

# CHIMERA: Protocol-Aware Rollback Recovery for Confidential Replicated Services

Tong Liu<sup>1</sup>, Xiaoqing Wen<sup>2</sup>, Ziwei Zhou<sup>3</sup>, Si Liu<sup>4</sup>, Jianyu Niu<sup>5</sup>, Cong Wang<sup>5</sup>, Yinqian Zhang<sup>1</sup>

<sup>1</sup>Southern University of Science and Technology <sup>2</sup>University of British Columbia <sup>3</sup>East China Normal University

<sup>4</sup>The Chinese University of Hong Kong, Shenzhen <sup>5</sup>City University of Hong Kong

## ABSTRACT

Confidential replicated services use Trusted Execution Environments (TEEs) to protect data and to run efficient crash-fault-tolerant (CFT) consensus as a Byzantine-fault-tolerant service. TEEs, however, do not guarantee freshness of persistent state: after a crash, a compromised host can replay an authentic but stale snapshot, causing a replica to forget votes or log progress, thereby threatening both linearizability and availability. Generic rollback defenses treat persistent state uniformly, making it difficult to simultaneously achieve high performance and fast replica re-entry.

We first systematize rollback-resilient recovery for confidential replicated services into four categories according to how they trade performance against availability. We then present CHIMERA, a protocol-aware recovery framework for leader-based confidential consensus. Our key insight is that recovery should follow protocol semantics: local, infrequently updated election metadata requires precise freshness protection, whereas high-volume replicated logs can be reconstructed from quorum state. CHIMERA therefore combines counter-bound metadata recovery with protocol-guided log recovery, including distinct follower and leader paths. We formalize the recovery contract and prove its safety and liveness properties. We demonstrate CHIMERA’s effectiveness by instantiating it with Braft, for which we formally verify the correctness of its design and implement it with Intel TDX. Our experimental results show that CHIMERA improves throughput and recovery latency over rollback-resilient baselines in both LAN and WAN deployments. We additionally demonstrate CHIMERA’s portability on ZooKeeper/Zab.

## PVLDB Reference Format:

Tong Liu<sup>1</sup>, Xiaoqing Wen<sup>2</sup>, Ziwei Zhou<sup>3</sup>, Si Liu<sup>4</sup>, Jianyu Niu<sup>5</sup>, Cong Wang<sup>5</sup>, Yinqian Zhang<sup>1</sup>. CHIMERA: Protocol-Aware Rollback Recovery for Confidential Replicated Services. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

## PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Artifacts2026/CHIMERA>.

## 1 INTRODUCTION

Replication is a foundation of distributed data management, allowing services to preserve consistency and availability despite machine failures and network uncertainty [34, 36, 62]. As replicas may

be compromised, Byzantine fault-tolerant (BFT) replication extends these guarantees to a bounded number of arbitrarily faulty replicas and has been explored for transactional databases, replicated ledgers, and other trustworthy data services [6, 14, 34, 86]. This robustness is costly: tolerating  $f$  Byzantine faults classically requires at least  $3f+1$  replicas, larger quorums, and additional communication, increasing deployment cost and limiting scalability [6, 14].

For sensitive workloads, fault tolerance alone is insufficient: data and computation must also remain confidential from the underlying infrastructure [34, 41, 67, 88]. Trusted Execution Environments (TEEs) offer a foundation for meeting both requirements. By isolating code, keys, and in-memory data from an untrusted host, TEEs have enabled confidential databases [67, 88], key-value stores and replicated ledgers [34, 41], and coordination services [12]. In replicated services, constraining protocol execution and protecting signing keys inside TEEs allow hardened crash-fault-tolerant protocols, such as Raft [62] and Zab [42], to tolerate  $f$  compromised hosts with  $2f+1$  replicas while preserving confidentiality [12, 20, 34, 80]. This construction, however, relies on each replica retaining its protocol history across crashes and restarts.

Commodity TEEs do not provide such state continuity. Sealing authenticates persisted state but does not guarantee its freshness: after a crash, a compromised host can restart a replica from an authentic but stale sealed state [54, 59, 65, 76, 80]. Such rollback does more than restore old data; it can restore stale protocol state that has already affected previous decisions. For example, a replica may lose its vote record and vote again, or rejoin with a log that omits previously committed history and violates quorum requirements. Rollback can therefore violate election safety, undermine committed-prefix preservation, or prevent a replica from safely rejoining the service, invalidating the consistency and availability guarantees of the underlying replication protocol.

Prior work obtains rollback resilience through four recurring strategies: Trusted Counters (TC), Diskless Crash Recovery (DCR), Rollback Fault Tolerance (RFT), and Reconfiguration (RC) [3, 23, 31, 35, 54, 59, 60, 65, 76, 80]. We classify them by how each trades performance, *i.e.*, the normal-path or recovery cost, against availability, *i.e.*, how soon a recovered replica can rejoin (§3). TC adds freshness coordination to protected updates. RFT enlarges replica and quorum sizes to mask rolled-back state. DCR reconstructs state from peers but may require conservative epoch advancement. RC replaces the old replica identity and resynchronizes its state. Each strategy can preserve safety. However, applying one dominant strategy to all persistent protocol state imposes its cost even on state for which a cheaper mechanism would suffice. The deeper limitation is therefore *state-agnostic recovery*: persistent state components encode different safety obligations and differ in whether they can be

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

reconstructed from other replicas. This paper asks: *How can a confidential replicated service achieve rollback-resilient recovery without facing the traditional tradeoff between performance and availability?*

Our key insight is that rollback recovery should follow protocol semantics. In leader-based replicated-log protocols, persistent state falls into two different classes. *Metadata*, such as an epoch and its vote-equivalent record, encodes a node-local non-equivocation obligation: once a replica has supported a leader in an epoch, that authority must not be revived. Because this local history may never have been observed by a quorum, other replicas cannot reliably reconstruct it. In contrast, the *log* records replicated global history. Its committed prefix is constrained by quorum intersection, log ordering, and the base protocol’s prefix rules, and can therefore be reconstructed without recovering an exact local disk image. Metadata thus requires a non-rollbackable local anchor, whereas logs can rely on protocol-guided distributed recovery. Although the concrete state components differ across protocols, the distinction between locally generated protocol obligations and quorum-reconstructed history is fundamental to replicated services.

Guided by this insight, we design CHIMERA, a protocol-aware rollback recovery framework for leader-based confidential replication. CHIMERA targets base protocols with monotonic epochs, a bounded vote-equivalent metadata record, quorum-preserved log prefixes, authenticated recovery, and static membership during recovery. For metadata, CHIMERA binds safety-critical protocol progress to a trusted counter. When the counter and sealed metadata match, the replica restores the metadata; when they differ, the counter anchors a safe epoch in which voting is disabled. For logs, CHIMERA queries authenticated replicas and reconstructs a safe prefix according to the quorum and freshness rules of the base protocol. Distinct follower and leader recovery paths reuse valid local state when possible, while allowing a crashed leader to resume after establishing a fresh epoch and recovering a safe log.

Realizing this separation is, however, nontrivial. First, counter increment and metadata sealing are not atomic: a crash between them is indistinguishable from malicious rollback. CHIMERA introduces a binding update that repairs this mismatch to a non-voting epoch (§6.2). Second, a crashed leader may leave an uncommitted suffix on only a subset of replicas. Leader recovery must re-establish leadership in a fresh epoch and select a log preserving every committed prefix (§6.3). Third, transferring a large log can dominate recovery time. CHIMERA uses recovered metadata, node roles, and local prefixes to fetch only the state required for safe re-entry.

To demonstrate CHIMERA’s effectiveness, we implement CHIMERA as a protocol-aware rollback recovery framework and instantiate its design on Braft/Raft using Intel TDX. We use the Braft instantiation to quantitatively compare CHIMERA against four representative rollback-resilient designs, covering normal-path performance, recovery cost, and TEE-related overhead under both LAN and WAN environments with up to 61 TDX-enabled cloud instances. We provide formal proofs that CHIMERA preserves the safety and liveness properties required by the underlying protocol and use Maude to further validate the Braft instantiation through model checking. To further assess the generality of the recovery abstraction beyond Raft-specific election semantics, we additionally instantiate CHIMERA on ZooKeeper/Zab, whose distinct epoch and election mechanisms provide a portability validation of the design.

**Contributions.** Overall, we make the following contributions:

- We systematize rollback-resilient recovery for confidential replicated systems into four categories based on their recovery trade-offs. We show that this limitation stems from applying a uniform recovery strategy to protocol states with different safety and reconstruction characteristics.
- We present CHIMERA, a protocol-aware recovery framework that protects non-reconstructible, node-local protocol state through counter-based metadata recovery while reconstructing replicated history through protocol-guided log recovery.
- We implement CHIMERA on Braft/Raft with Intel TDX, formally prove its recovery properties, model check the Braft instantiation in Maude, and quantify its normal-path and recovery costs under faults against representative rollback-resilient baselines.
- We demonstrate CHIMERA’s portability on ZooKeeper/Zab, showing that the design extends to a distinct leader-based CFT protocol with different epoch and election semantics.

## 2 BACKGROUND

### 2.1 TEEs and Their State Continuity

Trusted Execution Environments (TEEs) are hardware-supported execution environments that protect sensitive code and data. They achieve this protection through mechanisms such as memory encryption, hardware-enforced isolation, and remote attestation. Representative platforms include enclave-based TEEs such as Intel SGX [33], VM-based TEEs such as Intel TDX [37] and AMD SEV [43], and other hardware isolation architectures such as ARM TrustZone [5]. Compared with enclave-based TEEs, VM-based TEEs allow existing applications to benefit from TEE protection with minimal code changes and low additional overhead. We therefore build our implementation on VM-based TEEs.

TEEs have been adopted in a wide range of stateful applications, including blockchains [17, 47, 82, 83, 85, 87], trusted storage [12, 20, 41, 63], authentication rate limiting [76], and cloud computing services [3, 34, 35, 59, 68]. However, despite their popularity, TEEs still lack state continuity guarantees: rollback attacks can provide stale persistent data to revert a TEE to a prior state, undermining the security of applications built atop them.

**State Continuity.** State continuity of TEEs requires that when a stateful TEE application resumes execution after an interruption (e.g., reboots or system crashes), it preserves the consistency guarantees of its prior execution state [65]. This property is critical for applying TEEs in distributed systems, since a node’s ability to recover from crashes directly determines system reliability [29, 32, 70]. Existing TEE platforms provide sealing functionality, allowing TEE applications to encrypt and store their state on untrusted persistent storage. The sealing key can be configured to be accessible to all enclaves with the same MRENCLAVE or MRSIGNER [59].

**Rollback Attacks.** The sealing functionality can ensure the integrity of retrieved data, but does not provide freshness guarantees [67]. An adversary controlling the OS can roll back a TEE application to a previous state by providing it with stale data, resulting in a rollback attack [84]. These attacks break state continuity and undermine the security guarantees of TEEs for various stateful applications, particularly confidential BFT consensus systems. For

instance, if a node is rolled back after casting a vote, it may re-enter a previous state and cast the vote again. Such repeated voting can lead to equivocation, violating the safety properties of the protocol.

## 2.2 Confidential BFT Consensus

Confidential BFT consensus ports CFT consensus protocols into TEEs to provide BFT services. Representative systems include Signal’s SVR3 [20, 74] and Azure’s CCF [7, 34]. Academic designs include Engraff [80], protected Hyperledger Fabric [11], and SecureKeeper [12]. Among them, SVR3, CCF, and Engraff are built on Raft [62], whereas SecureKeeper uses Zab [42]. Raft and Zab are well-established leader-based CFT consensus protocols, which are leveraged in our design.

- **Epoch.** The leader-based consensus protocol partitions time into logical units called epochs (referred to as terms in Raft). For each epoch, at most one node, called the leader, is elected and agreed upon by a quorum. When the leadership needs to change, the epoch is incremented to reflect the transition.
- **Leader Election.** When nodes detect that the leader is unavailable, they initiate a leader election to preserve liveness. Specifically, each node casts at most one vote per epoch and persists this voting behavior to prevent double voting. A candidate node becomes a leader when it receives votes from the majority of nodes. A non-leader node is called a follower. In this paper, we refer to the persisted epoch and voting behavior as metadata.
- **Log.** The log is a sequence of client transactions maintained by nodes, where each entry contains one or a batch of transactions. Each entry is uniquely identified by an epoch number and a monotonically increasing index. Nodes commit entries in a contiguous sequence sorted by index without gaps. Unlike metadata, logs are replicated across nodes following consensus rules.

## 3 ROLLBACK RECOVERY DESIGN SPACE

Rollback recovery schemes differ in their recovery strategies and the guarantees they provide for safe re-entry after rollback. We establish a common scope and characterize these schemes based on their recovery precision and their impact on performance and replica availability (§3.1). Based on this characterization, existing approaches are classified into four families (§3.2), revealing different tradeoffs in the rollback recovery design space. These observations motivate the design requirements for CHIMERA (§3.3).

### 3.1 Scope and Characterization Criteria

We study existing rollback recovery schemes for TEE-enabled distributed systems, where a crashed replica may restart from an outdated persistent state due to rollback attacks. In particular, we study how to restore a replica after such rollback while preserving consistency with the distributed protocol execution.

Despite different system settings, existing rollback recovery schemes generally follow a common recovery process: they prepare necessary information during normal execution and use it after rollback to recover the replica and resume protocol execution. We characterize these schemes by their recovery precision and analyze how the resulting design choices affect the tradeoff between performance and availability.

- **Recovery precision.** Recovery precision captures how accurately a scheme can determine a safe state for a rolled-back replica. More precise recovery requires richer recovery evidence and more elaborate preparation or recovery procedures.
- **Performance.** Performance captures the overhead introduced by rollback recovery mechanisms during normal execution and recovery. Maintaining more recovery evidence or performing more complex recovery procedures improves recovery precision but increases runtime overhead.
- **Availability.** Availability captures how quickly a recovered replica can resume normal protocol operation after rollback. Imprecise recovery may leave uncertainty about whether the replica can safely participate in the protocol. The replica may therefore need to wait for additional confirmation before rejoining, resulting in temporary availability degradation.

Rollback recovery schemes balance recovery precision against performance overhead and availability. More precise recovery typically requires additional preparation and recovery overhead, whereas less precise recovery may leave greater uncertainty after rollback and delay replica availability. Existing approaches therefore occupy different points in the rollback recovery design space.

### 3.2 Rollback Recovery Taxonomy

Based on the above characterization criteria, we classify existing rollback recovery schemes into four categories: Trusted Counter (TC), Diskless Crash Recovery (DCR), Rollback Fault Tolerance (RFT), and Reconfiguration (RC), as summarized in Table 1. These categories represent different design choices in the rollback recovery design space, leading to different tradeoffs between performance and replica availability.

**Trusted Counter (TC).** TC achieves high recovery precision by maintaining additional local recovery information, but introduces overhead on the normal execution path [54, 59, 65, 66, 76, 80]. It uses tamper-resistant counters whose values cannot be reverted once incremented [69]. For each state update, TC binds the protected state to the counter value for later freshness verification. During recovery, the replica compares the counter value associated with the recovered state against the current counter value. The above design choice leads to the following limitations:

- *High update overhead.* Each state update requires counter operations, introducing substantial normal-path overhead. Hardware-backed counters incur tens of milliseconds for update and verification, while software-based counters require additional communication rounds [54, 59, 66, 76].
- *Atomicity dilemma between counter update and state persistence.* Counter advancement and state sealing cannot be atomic. The *inc-then-store* scheme preserves safety but may hinder recovery, whereas the *store-then-inc* scheme improves availability but enables rollback [54, 59, 65, 66, 76, 80]. See Appendix C for details.
- *Limited recovery guidance.* TC provides freshness checking by comparing counter values, but the counter alone does not determine how to recover from rollback.

**Diskless Crash Recovery (DCR).** DCR reduces recovery overhead by leveraging existing replication state, but sacrifices availability due to its limited recovery capability for replica-local protocol

**Table 1: Rollback-resilient solutions.**

| Approach       | Recovery Pattern | Recovery Completeness | Performance Overhead | Availability Degradation |
|----------------|------------------|-----------------------|----------------------|--------------------------|
| TC             | Local            | High                  | High                 | Low                      |
| DCR            | Distributed      | Medium                | Low                  | High                     |
| RFT            | Local            | High*                 | High                 | Low                      |
| RC             | Distributed      | Low                   | Low                  | High                     |
| <b>CHIMERA</b> | Hybrid           | High                  | Low                  | Low                      |

\* RFT enables a node to recover its state, but cannot prevent state rollback.

state [15, 46, 48, 55]. DCR recovers a replica’s state from the in-memory state of other replicas in the system. It leverages existing replication mechanisms to recover redundant state, such as logs, without introducing additional overhead on the normal path. However, DCR exposes the following limitations:

- *Limited recovery coverage.* DCR can recover replicated state from other replicas, but cannot recover replica-local protocol state, such as voting-related metadata. Without sufficient information, a recovering replica cannot directly determine a safe execution state and may require conservative recovery before resuming protocol participation, resulting in availability degradation.
- *Protocol-specific recovery logic.* The conservative recovery rules are determined by the semantics of the protocol. Since different protocols define different safety boundaries for recovery, the recovery procedure requires protocol-specific customization.

**Rollback Fault Tolerance (RFT).** RFT tolerates rollback faults by adjusting quorum requirements or system size rather than recovering rolled-back state. RR [23] modifies read/write quorum sizes to preserve intersection with up-to-date nodes, while FlexiBFT [31] increases the system size to  $3f + 1$  nodes following classical BFT assumptions. By avoiding explicit recovery of rollback-sensitive state, RFT enables fast replica re-entry, but requires stronger quorum or deployment configurations:

- *Scalability limitation.* Increasing quorum or node size increases the cost of consensus operations. For example, atop Braft, the performance gap between  $2f + 1$  and  $3f + 1$  configurations grows from around 10% for small  $f$  to roughly 20% for  $f = 20$  (§7.3). Larger node configurations also conflict with TEE-aided designs that aim to reduce deployment size [10, 19].

**Reconfiguration (RC).** RC avoids direct rollback recovery by treating a recovered replica as a new participant and adding it back to the system through membership changes. Notable confidential BFT protocols, such as CCF [34] and Recipe [30], adopt this approach. By avoiding recovery-specific mechanisms, RC introduces no additional overhead during normal execution, but may degrade availability due to the cost of rejoining the system:

- *Expensive membership changes.* To preserve safety, RC requires replicas to coordinate membership updates and apply them consistently. Such coordination introduces additional consensus steps and may delay concurrent protocol execution.
- *Expensive state synchronization.* A rejoining replica must synchronize application state from existing replicas before resuming execution [34]. For applications with large state, such as blockchains,

this process may require transferring hundreds of gigabytes of data and take hours or even days [26, 45].

### 3.3 Implications for Protocol-Aware Recovery

Existing approaches explore different tradeoffs between performance and availability, but none of them achieves a satisfactory balance. This limitation stems from applying a uniform recovery mechanism to heterogeneous persistent state.

The key observation is that persistent state has different recovery characteristics. This observation motivates a protocol-aware recovery design that matches recovery mechanisms with the characteristics of different state types. This leads to three requirements:

- *R1 (Local evidence where peers cannot help):* Replica-local state requires precise recovery evidence with bounded overhead.
- *R2 (Reuse protocol redundancy):* State already replicated by the protocol should be leveraged for recovery, avoiding unnecessary recovery mechanisms and normal-path overhead.
- *R3 (Timely re-entry after recovery):* A replica should resume participation promptly once sufficient safety conditions are satisfied, without unnecessary recovery delays.

## 4 PROBLEM STATEMENT

### 4.1 System Model

We consider a static configuration of  $n = 2f + 1$  replicas  $\{p_1, \dots, p_n\}$ , following prior confidential replicated services [34, 80]. Each replica executes a leader-based CFT protocol inside a TEE; without rollback, this protected base protocol is assumed to satisfy its ordinary safety and liveness properties. A finite set of clients submits transactions over encrypted and authenticated channels.

We assume a Public Key Infrastructure (PKI). Replica  $p_i$  has a key pair  $(pk_i, sk_i)$ , and  $sk_i$  is available only to its TEE. We write  $m_{\sigma_i}$  for a message signed with  $sk_i$ . Recovery messages additionally carry a fresh nonce and are accepted only over an authenticated channel, preventing a host from forging another replica’s state or replaying an old recovery response.

**Adaptive Fault Budget.** The adversary  $\mathcal{A}$  may adaptively change the set of corrupted hosts over time, while the number of faulty replicas is bounded by  $f$  at any point. A replica is considered faulty if it is corrupted, crashed, or undergoing recovery. When the adversary releases a corrupted host, the corresponding replica remains faulty and continues to consume the fault budget until CHIMERA completes its recovery, after which it becomes correct and the adversary may corrupt another replica. Correct replicas faithfully execute the protected protocol and remain available.

On a corrupted host,  $\mathcal{A}$  controls the OS and can delay, drop, reorder, or inject host-level I/O; start and stop the TEE; and supply an earlier *authentic* sealed snapshot during restart [54, 65, 75–77, 80]. A rollback therefore does not modify enclave code or keys. Nor can the adversary roll back a correct replica: a compromised or recovering replica stays within the faulty budget until it has re-established a safe state.

**TEE Security Boundary.** We rely on TEE code integrity and key confidentiality, unforgeable authenticated channels, trusted-counter integrity and monotonicity, and single-instance execution

for each replica identity. If code or counter integrity fails, an attacker can bypass recovery checks or reuse votes; if key confidentiality fails, it can forge protocol messages; and if two concurrent instances share one identity, they can equivocate. Any of these violations can break CHIMERA’s safety. Concurrent cloning and micro-architectural side channels are therefore outside our model [16, 54, 59, 72, 78]; deployment mechanisms such as TPM-based launch binding [76] and side-channel-hardened libraries [61, 73] are complementary, not guarantees supplied by CHIMERA.

**Network Model.** We adopt the partially synchronous network model [24], which is commonly used in consensus [14, 27, 40, 89]. In this model, there is an established bound  $\Delta$  and an undefined Global Stabilization Time (GST). After the GST point, every message sent between two correct replicas is delivered within  $\Delta$ . That is, the system behaves *synchronously* following the GST. Liveness is guaranteed after GST.

## 4.2 Recovery Contract

The protected base protocol proceeds through monotonically increasing epochs and replicates an ordered transaction log. In every epoch, a replica grants at most one vote (or the protocol-specific equivalent), and a candidate becomes leader after collecting a majority. We model the persistent election metadata at  $p_i$  as  $md_i = (epoch_i, vote_i)$ , where  $vote_i$  is either a candidate identity,  $\perp$  (not yet voted), or disabled (voting disabled for that epoch). Concrete protocols may encode this record differently; for example, Zab uses accepted and current epochs rather than Raft’s *votedFor* field.

The leader batches client transactions into log entries tagged by an epoch-index pair  $(ep, idx)$  and replicates them to followers. An entry becomes committed after a majority has acknowledged it under the base protocol’s prefix and up-to-date-log rules. CHIMERA requires a committed entry to remain available at a majority at commitment time, but it does not require every replica to synchronously flush its copy to local disk: under the concurrent fault bound, at least one correct quorum member retains the entry for protocol-guided reconstruction.

As in standard state-machine replication [14], the base service provides **safety** (linearizability: no two correct replicas commit different entries at one index) and **liveness** (submitted transactions eventually commit after GST). The definitions below are not a replacement for that complete consensus specification. They are the recovery conditions that, composed with a correct base protocol, preserve its guarantees across rollback.

**Recovery under TEE rollbacks.** A safe recovery must restore sufficient protocol state to preserve election safety and committed-log invariants, rather than necessarily reconstructing the exact pre-crash local state. We state these obligations as three properties. The first two preserve safety:

**DEFINITION 1 (ELECTION SAFETY).** *For any epoch  $ep$  and any two distinct nodes  $p_i$  and  $p_j$ , it is impossible for both  $p_i$  and  $p_j$  to be elected leaders in  $ep$ .*

**DEFINITION 2 (LEADER COMPLETENESS).** *For any log entry  $en$  committed at index  $idx$  in epoch  $ep$ , every leader elected in a later epoch  $ep' > ep$  must contain  $en$  at index  $idx$  in its log.*

Election safety guarantees a unique leader for each epoch, while leader completeness carries every committed transaction into later leader logs. For liveness, a replica that is no longer under adversarial control must be able to rejoin:

**DEFINITION 3 (RECOVERY LIVENESS).** *For any uncorrupted node  $p_i$ , if  $p_i$  starts recovery and remains uncorrupted, then  $p_i$  eventually completes recovery and resumes participation in the protocol.*

## 5 CHIMERA OVERVIEW

We present an overview of CHIMERA, a protocol-aware recovery framework for confidential BFT consensus. As discussed in §3.3, existing rollback recovery schemes struggle to balance performance and availability because they apply uniform mechanisms to heterogeneous persistent state. To satisfy the requirements identified in §3.3, CHIMERA first characterizes the persistent state maintained by leader-based consensus protocols (§5.1) and then applies recovery strategies specialized for each state type (§5.2).

This framework improves the performance–availability trade-off of rollback recovery by matching recovery mechanisms with state characteristics, while preserving *election safety*, *leader completeness*, and *recovery liveness*.

### 5.1 Characterizing System State

For protocols satisfying this contract, three types of state may be externalized: metadata, logs, and snapshots. Snapshots accelerate synchronization but are derived from the committed log, so their rollback does not independently violate safety. We focus on the two safety-critical states—metadata and logs—summarized in Table 2.

**Metadata.** Metadata denotes a set of safety-critical control variables that govern leader election and epoch transitions. Metadata is updated infrequently and generally changes during leader transitions. It typically includes the current epoch and the node’s voting record within that epoch. For example, in Raft, metadata consists of *currentTerm* (the epoch identifier) and *votedFor* (the candidate voted for in that epoch). The epoch identifier allows nodes to recognize larger epochs and reject stale leadership attempts. The voting record prevents double voting, ensuring election safety.

Two properties make metadata distinct for recovery. First, metadata is node-local: it records a node’s own view of consensus. As a result, other replicas cannot reliably reconstruct the recovering node’s exact epoch or vote. Therefore, recovery must balance safety and availability: ensuring freshness of metadata preserves election safety, but entering a higher epoch may temporarily reduce availability. This trade-off makes metadata well-suited for TC-based local freshness protection, which enables precise, node-local recovery. Second, metadata can be stored in a register-like form. In particular, the epoch increases monotonically during leader transitions, which aligns naturally with the TC design. Moreover, the low update frequency of metadata ensures that introducing TC incurs minimal additional overhead.

**Log.** Each node maintains an ordered log. An entry is committed once it has been accepted by a majority under the base protocol’s rules. Leader completeness preserves committed entries across subsequent leaders. Recovery therefore needs a safe committed prefix to remain available within the quorum, not a fresh local disk copy at every replica. This distinction permits background persistence as

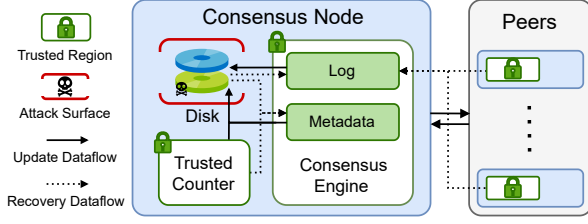


Figure 1: Architecture of CHIMERA.

long as the concurrent-fault bound leaves a correct quorum member from which lost entries can be reconstructed.

Logs differ from metadata in distribution and form. Unlike metadata, which is node-local, logs are replicated across multiple nodes in a structured way. Therefore, we can leverage the replicas to reconstruct a safe log during recovery. Moreover, logs are large and stored on disk, making it infeasible to maintain a direct mapping to a trusted counter. Version numbers can be used to detect inconsistencies, but they do not enable full recovery of log contents.

## 5.2 Protocol-Aware State Recovery

We leverage protocol-level knowledge, *i.e.*, the inherent characteristics of logs and metadata, to carefully tailor recovery strategies, as illustrated in Fig. 1.

**Metadata Recovery.** CHIMERA protects metadata using a trusted counter (TC), addressing the requirement of preserving freshness for non-reconstructible local state (R1). The TC enables a recovering node to accurately verify sealed metadata without relying on other replicas. This approach provides precise recovery for non-reconstructible metadata without availability degradation.

Raw TC alone, however, is insufficient: counter increments and metadata sealing are not atomic, so a mismatch may indicate either a rollback or a crash during the update window. To address this issue, CHIMERA introduces a binding-update design that associates the epoch with the counter value. When the counter and epoch match, the node safely restores the sealed metadata; otherwise, the counter provides a safe epoch anchor, enabling recovery without conservative epoch skipping.

**Log Recovery.** For logs, CHIMERA relies on protocol-guided cluster recovery rather than per-entry TC protection (R2). Consensus replication and the concurrent-fault bound leave committed entries available at a correct quorum member. During recovery, a node queries authenticated replicas and reconstructs a log that includes the committed prefix according to the base protocol’s selection rule.

CHIMERA also tailors recovery to the node’s role to improve availability (R3). A recovering follower can reuse valid local state and avoid unnecessary log transfer during catch-up. A recovering leader, in contrast, can resume service after reconstructing a safe log without waiting for a new leader election. By leveraging both protocol-level guarantees and node role information, CHIMERA enables safe and efficient log recovery while preserving availability.

Table 2: The characteristics of metadata, logs, and snapshots.

| State Type | Safety Critical | Recovery Requirement | System Redundancy | Update Frequency |
|------------|-----------------|----------------------|-------------------|------------------|
| Metadata   | ✓               | Precise              | ✗                 | Low              |
| Log        | ✓               | Loose                | ✓                 | High             |
| Snapshot   | ✗               | Loose                | ✗                 | Low              |

## 6 CHIMERA DESIGN

### 6.1 Data Structures and Interfaces

**Data Structures.** CHIMERA handles the update and recovery of metadata and logs separately.

1) *Metadata.* The recovery interface models node  $p_i$ ’s metadata as  $md_i = (ep_i, vote_i)$ , with the vote states defined in §4.2. This is an abstract interface rather than a claim that every implementation stores two literal fields. Raft maps it to *currentTerm* and *votedFor*, while Zab realizes the same obligations through *acceptedEpoch* and *currentEpoch*. Any vote-equivalent operation is executed only after the corresponding metadata update has been sealed.

2) *Log.* Each node  $p_i$  maintains a local log  $log_i$ , which consists of a sequence of entries  $en_j$  indexed by  $j$ . A log entry is identified by its epoch and index; we refer to this pair  $(ep, j)$  as the entry’s *epoch-index tag*. We compare epoch-index tags lexicographically:  $(ep_1, idx_1) > (ep_2, idx_2)$  iff  $ep_1 > ep_2$ , or  $ep_1 = ep_2$  and  $idx_1 > idx_2$ .

**Interfaces.** CHIMERA relies on sealing interfaces for protecting states stored in external storage, and on TC interfaces for maintaining freshness across crashes.

1) *Sealing interfaces.* TEEs provide two sealing interfaces:

- **Seal**( $data, h$ ): Encrypt  $data$  with the TEE-internal sealing key and then store the associated encrypted data at the location identified by handle  $h$  in external storage.
- $data \leftarrow$  **UnSeal**( $h$ ): Retrieve the sealed data from an external storage with the handle  $h$ , decrypt it, and return the associated plaintext  $data$ .

2) *TC interfaces.* We use two TC operations:

- $ctr \leftarrow$  **IncTC**( $k$ ): Atomically increment the counter identified by key  $k$  by one and return the new value  $ctr$ .
- $ctr \leftarrow$  **ReadTC**( $k$ ): Return the current value  $ctr$  of the counter identified by key  $k$  without modifying it.

Each node maintains two trusted counters for different recovery purposes:

- $TC_{md}$  records metadata updates. Its value is bound to the epoch in sealed metadata, providing a recovery anchor when the sealed metadata and counter value mismatch.
- $TC_{role}$  tracks a node’s role for recovery. It is incremented whenever the node switches between follower and leader, with odd values indicating leader and even values indicating follower. During recovery, the counter allows the node to determine its previous role and select the corresponding recovery path.

**Initialization.** Each node boots from a hard-coded genesis configuration and initializes with default metadata (*i.e.*,  $ep = 1$  and

$vote = \perp$ ) and an empty log. The configuration information, including key pairs, is stored on local disks in an encrypted and authenticated form.

In this work, we consider a static configuration: a recovering node can retrieve the configuration to obtain its own key pairs and the public keys of other nodes for communication. Dynamic reconfiguration is discussed in §9.

The trusted counters are initialized to match the starting system state:  $TC_{md}$  is set to 1 to align with the initial epoch, while  $TC_{role}$  is set to 0, encoding the initial follower role. These initializations establish a consistent initial state for metadata and role tracking.

## 6.2 Metadata Recovery Mechanism

**6.2.1 Metadata Update.** CHIMERA protects metadata with TC, since metadata is node-local and requires more precise recovery to gain higher availability. For each metadata update, the node first increments  $TC_{md}$  and then seals the metadata together with the counter value, following an *inc-then-store* scheme [76] (Fig. 2a).

TC provides strong evidence for detecting stale metadata, but a raw TC is insufficient: if a crash occurs after the counter is incremented but before the updated metadata is sealed, recovery observes a counter mismatch and cannot distinguish between a benign crash and a rollback. CHIMERA resolves this ambiguity by binding the epoch to  $TC_{md}$ . The key rule is to advance both values in lockstep: every metadata update increases the epoch by exactly one and performs one **IncTC**(md). This design turns  $TC_{md}$  into a record of epoch transitions. When the counter value matches the sealed metadata, the node can safely restore the sealed metadata from persistent storage. In the event of a mismatch, CHIMERA uses the counter value as the recovered epoch. To maintain safety, the node disables voting in this epoch due to uncertainty about its previous votes.

The *target epoch*  $ep^*$  is the epoch carried by the request. When  $ep^*$  is larger than the node’s current epoch  $ep$ , this node should advance its epoch to the target epoch  $ep^*$ . CHIMERA does not change that election logic. It replaces the persistence step used when the base protocol advances from  $ep$  to  $ep^*$ . The node executes the following loop until it reaches  $ep^*$ :

- ① Invoke **IncTC**(md) to advance the trusted counter  $TC_{md}$  by one step and obtain the new value  $ctr$ .
- ② Update the metadata: increment  $md.epoch$  by 1 and set  $md.vote$  according to the request type. For all intermediate epochs, set  $md.vote$  to non-voting. When  $md.epoch$  reaches the target  $ep^*$ , set  $md.vote$  to the selected candidate if the request is a vote request; otherwise, it remains non-voting.
- ③ Persist metadata: call **Seal**( $md|ctr, h_{md}$ ) to store the updated metadata along with the counter value.
- ④ Send vote reply: for vote requests, the node sends the reply when  $md.epoch$  reaches  $ep^*$ . The reply is sent after the corresponding metadata has been sealed to ensure consistency.

A multi-epoch advance process is executed as a sequence of single-epoch updates. All intermediate epochs are marked non-voting. The final epoch records a candidate only if the triggering request is a granted vote. This binding update keeps  $md.epoch$  synchronized with  $TC_{md}$  and prevents the node from casting a different vote in any previous epoch.

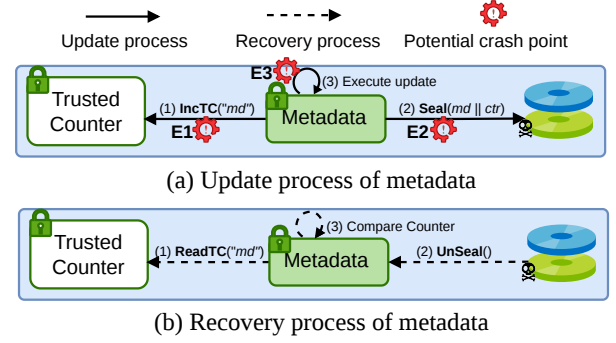


Figure 2: Recovery design of metadata.

**6.2.2 Metadata Recovery.** As shown in Fig. 2b, metadata recovery follows the update scheme. The node first verifies the consistency between the sealed metadata and  $TC_{md}$ . If the sealed counter value matches the current counter value, the node recovers from the sealed state. Otherwise, it recovers to a safe epoch without risking double voting. The binding-update design binds  $TC_{md}$  to the current epoch, maintaining a correspondence between the counter value and epoch. This correspondence provides a reliable anchor for safe recovery. The recovery process then proceeds as follows:

- ① Load sealed metadata: the node invokes **UnSeal**( $h_{md}$ ) to retrieve  $md'$  together with the sealed counter value  $ctr'$ .
- ② Read current counter: the node calls **ReadTC**(md) to obtain the current counter value  $ctr$  for freshness verification.
- ③ Check counter match: if  $ctr = ctr'$ , the sealed metadata is fresh. The node sets  $md \leftarrow md'$  and finishes recovery.
- ④ Handle mismatch: if  $ctr \neq ctr'$ , the sealed metadata may be outdated. Then the node starts metadata repair with  $ctr$ .
- ⑤ Repair metadata: the node reconstructs metadata by setting  $md.epoch \leftarrow ctr$  and  $md.vote$  to non-voting. The node then resumes execution using this repaired metadata.

**6.2.3 Crash Window Analysis.** The inc-then-store update is not atomic. Crashes can occur in three intervals, as illustrated in Fig. 2a: before incrementing  $TC_{md}$  (E1); after incrementing  $TC_{md}$  but before sealing the metadata (E2); and after sealing the metadata but before sending the vote reply or completing the triggering operation (E3). Our analysis shows that each crash falls into one of two cases. In the match case, the sealed metadata can be safely restored. In the mismatch case, the node repairs to a non-voting epoch.

- **E1 Crash.** At E1, the node has prepared an update, but neither the epoch nor the  $TC_{md}$  has changed. Since nothing has been written to persistent metadata, there is no mismatch. Recovery can safely restore the previous metadata, and no actions or effects have been produced.
- **E2 Crash.** At E2,  $TC_{md}$  has advanced by one epoch, but the corresponding metadata has not yet been sealed. This creates a mismatch between the counter and the sealed metadata. This mismatch is indistinguishable from an actual rollback, rendering detection unreliable. The binding update design resolves this issue by using the counter value as the recovered epoch. Since no sealed metadata is available, the node must disable voting in that epoch to prevent casting or resending votes with incomplete metadata.

- **E3 Crash.** At E3, both  $TC_{md}$  and the sealed metadata have been updated. If no rollback occurred, the counter and sealed metadata match, allowing the node to safely restore the sealed metadata. With the complete metadata, any previously recorded voting behavior can also be safely restored. If a rollback occurred, a mismatch between the counter and the metadata will be observed. Recovery then uses the counter value to restore the epoch and marks the node as non-voting.

### 6.3 Log Recovery Mechanism

A recovering node begins log recovery once metadata recovery has established a safe epoch. Unlike metadata, logs are replicated according to consensus rules, allowing CHIMERA to leverage protocol redundancy for recovery. The role counter  $TC_{role}$  allows a recovering node to determine its role at the time of the crash, and CHIMERA uses different recovery paths for followers and leaders. Alg. 1 summarizes both paths. Every request and response is authenticated by the sender's TEE and binds the sender identity, epoch, role, log tag, and a fresh request nonce; the pseudocode writes this nonce as *non* and omits signatures for readability.

**6.3.1 Log Recovery for Followers.** Committed entries must be durably stored on a quorum of replicas. A recovering follower, therefore, needs to catch up with a safe leader. Generic DCR-style recovery may collect  $f+1$  replies to identify the latest leader and ensure that entries still being replicated are not lost. In CHIMERA, metadata recovery already restores a safe epoch for the follower. Therefore, a recovering follower can synchronize its log from any leader whose epoch is no lower than its own.

The follower recovery path leverages this property to reduce message overhead. The follower unseals its local log and determines its last-entry tag, then broadcasts a FOLLOWERRECOVER message with its recovered epoch and last-entry tag (Alg. 1, L6–L12). An eligible leader with no lower epoch replies with the missing suffix, allowing the follower to reuse valid local entries and fetch only the missing entries (Alg. 1, L18–L26). If tags mismatch, the follower overwrites inconsistent entries with the leader's log.

The follower accepts the first authenticated reply from an eligible leader whose epoch is no lower than its recovered epoch. No quorum certificate is required: TEE integrity authenticates the leadership claim, and leader completeness guarantees that a valid leader's log contains all committed entries needed for recovery. Therefore, the first eligible leader response is sufficient for recovery. Subsequent replies are ignored.

**6.3.2 Log Recovery for the Leader.** The leader recovery path is designed to minimize the system's unavailability following a leader crash. If the leader crashes, the system remains unavailable until the failure is detected by heartbeat and a new leader is elected. In CHIMERA, the crashed leader can resume service after recovery.

The leader first checks  $TC_{role}$  to determine whether it was a leader before the crash. It then advances the epoch to  $ep_i + 1$  and broadcasts a LEADERRECOVER message (Alg. 1, L13–L16). A node replies only if it can vote in epoch  $ep_i + 1$ . Before sending its reply, the node records that it has voted for the leader at epoch  $ep_i + 1$ . In this way, every follower that contributes to the recovery quorum will not vote for another candidate in the same epoch (Alg. 1,

---

#### Algorithm 1 Log recovery at node $p_i$

---

```

1:  $resp \leftarrow 0$  // the number of REPLYRECOVER messages received
2:  $tag_{max} \leftarrow \perp$  // the most up-to-date last-entry tag observed
3:  $src \leftarrow \perp$  // the ID of the node selected as the log source
4:  $done \leftarrow \text{false}$  // whether follower recovery has completed
5:
6: upon (LOGRECOVER,  $h_{log}$ ):
7:    $log_i \leftarrow \text{UnSeal}(h_{log})$ 
8:    $ep_i \leftarrow$  recovered metadata epoch of  $p_i$ 
9:    $last_i \leftarrow$  last-entry tag of  $log_i$ 
10:   $roleCtr_i \leftarrow \text{ReadTC}(role)$ 
11:  if  $roleCtr_i$  is even then
12:    broadcast (FOLLOWERRECOVER,  $i, ep_i, last_i, non$ )
13:  else
14:     $ep_i \leftarrow ep_i + 1$ 
15:    broadcast (LEADERRECOVER,  $i, ep_i, last_i, non$ )
16:  end if
17:
18: upon receiving (FOLLOWERRECOVER,  $j, ep_j, last_j, non$ ):
19:  if  $role_i \neq \text{leader}$  or  $ep_i < ep_j$  then return end if
20:   $suffix \leftarrow$  log entries after  $last_j$ 
21:  send (REPLYLOG,  $ep_i, suffix, i, non$ ) to  $p_j$ 
22:
23: upon receiving (REPLYLOG,  $ep, suffix, j, non$ ):
24:  if  $non$  is invalid or  $done$  or  $ep < ep_i$  then return end if
25:  append valid  $suffix$  to  $log_i$ 
26:   $done \leftarrow \text{true}$ 
27:
28: upon receiving (LEADERRECOVER,  $j, ep_j, last_j, non$ ):
29:  if  $ep_i > ep_j$  or ( $ep_i = ep_j$  and  $vote_i \neq \perp$ ) then
30:    return
31:  end if
32:  if  $ep_i \leq ep_j$  then
33:    // use the binding update of metadata
34:     $ep_i \leftarrow ep_j$ 
35:     $vote_i \leftarrow \text{disabled}$ 
36:  end if
37:   $last_i \leftarrow$  last-entry tag of  $log_i$ 
38:  send (REPLYRECOVER,  $ep_i, last_i, i, non$ ) to  $p_j$ 
39:
40: upon receiving (REPLYRECOVER,  $ep, last, j, non$ ):
41:  if  $non$  is invalid then return end if
42:   $resp \leftarrow resp + 1$ 
43:  if  $last$  is more up-to-date than  $tag_{max}$  then
44:     $tag_{max} \leftarrow last$ ;  $src \leftarrow j$ 
45:  end if
46:  if  $resp = f + 1$  then
47:    synchronize log with  $p_{src}$  and resume proposing
48:  end if

```

---

L28–L38). The recovery process itself acts as an election in this epoch: each replica that replies records its vote for the recovering leader through a binding update. Therefore, no replica contributing to recovery can later vote for another candidate in the same epoch, preserving election safety.

The recovering leader collects  $f+1$  replies and selects the log with the most up-to-date last-entry tag. This quorum is sufficient to preserve all committed entries. By quorum intersection, any entry committed before the crash must appear in at least one of the

replies. Selecting the log with the latest last-entry tag ensures that all committed entries are included (Alg. 1, L40–L48).

If the recovering leader resumes in the old epoch without advancing, entries that were not yet committed could cause inconsistencies. Depending on which followers respond, such entries may or may not be included in the log selected from the  $f+1$  responses. If the leader resumes in the old epoch after losing these entries, it could propose conflicting entries at the same index under the same epoch-index tag. Moving to a new epoch eliminates this ambiguity. After synchronization, the leader extends the recovered log in the new epoch, and any unrecoverable, uncommitted suffix from the old epoch is safely discarded.

**6.3.3 Log Recovery Acceleration.** CHIMERA further reduces recovery and normal-case overhead through two optimizations enabled by protocol-guided log recovery.

**Unblocking Log Recovery.** If the same leader remains active throughout the follower’s recovery, it tracks the next entry index for each follower. For follower  $p_i$ , this index is denoted as  $idx_i$ . After unsealing its log from persistent storage,  $p_i$  can complete recovery once it has caught up to  $idx_i$ , without synchronizing with the leader’s latest log. This mechanism accelerates recovery by eliminating unnecessary synchronization.

**Background Log Persistence.** CHIMERA moves log persistence off the commit critical path to reduce synchronous I/O during normal-case replication. Under high load, a node keeps newly appended log entries in memory and flushes them to disk in the background. This design is safe because log recovery relies on the cluster rather than the persistent storage as the sole source. By contrast, some systems require every committed log entry to be durably written to local storage. Such systems cannot use background persistence. Losing unflushed entries in these cases would compromise safety.

## 6.4 Correctness Reasoning

We give a proof sketch for the three recovery properties. The complete assumptions, invariants, lemmas, and dependency-ordered proofs appear in Appendix A.

**Proof Sketch of Election Safety.** Counter-epoch binding ensures that the recovered epoch never decreases, while mismatch repair prevents further support when a replica’s prior action is uncertain. An ordinary vote and a `REPLYRECOVER` both consume the replica’s per-epoch support: each is persisted before transmission, and each replica can issue at most one such support in an epoch. Any two ordinary-election or Recovery Election majorities in a system of  $2f+1$  replicas must intersect. Their common replica would therefore have to support two different leaders in the same epoch, contradicting the binding rule. Advancing a recovering leader to a fresh epoch further prevents different log entries from reusing the same (*epoch, index*) tag after an uncommitted suffix is lost.

**Proof Sketch of Leader Completeness.** At commitment, every committed entry is present in a majority. Any later election or recovery majority intersects this commit quorum, ensuring that at least one response contains the committed prefix. The base protocol’s safe-log rule then selects a recovery log that preserves this prefix, and the new leader appends only after it. By induction over subsequent epochs, every committed entry is therefore preserved in the log of every later leader.

**Proof Sketch of Recovery Liveness.** Metadata recovery completes after a bounded number of counter read and sealing operations. After GST, a recovering follower eventually reaches an eligible leader and fetches a valid suffix. A former leader either obtains a Recovery Election majority and resumes in a fresh epoch, or loses that opportunity to an ordinary election. In the latter case, it eventually continues through follower recovery. Thus, under partial synchrony, every uncorrupted recovering replica eventually completes recovery and re-enters the protocol.

## 7 IMPLEMENTATION AND EVALUATION

We implement CHIMERA-B by modifying Braft [8], which uses Raft as its underlying leader-based CFT consensus protocol. The implementation runs inside Intel TDX and integrates trusted metadata protection with protocol-guided log recovery. The Braft instantiation is also formally verified using Maude; details of the model and verification results are provided in Appendix D.

We compare CHIMERA-B against representative baselines derived from the four recovery strategy families in §3. For a consistent comparison, all variants running inside Intel TDX perform persistent disk I/O through the same sealing mechanism. We organize the evaluation around three questions:

- Q1:** What is the normal-path cost of CHIMERA, and how does it scale across LAN and WAN deployments? (§7.3)
- Q2:** How quickly can CHIMERA recover a failed replica and restore participation compared with prior recovery approaches? (§7.4)
- Q3:** Where do CHIMERA’s performance and recovery gains come from, and what overhead is introduced by TEE-related operations and individual optimizations? (§7.5)

We additionally port CHIMERA to ZooKeeper/Zab to assess the *portability* of the recovery abstraction beyond Raft-based protocols. The results show that the design can be adapted to Paxos-family CFT protocols with different epoch and election semantics, with ablation results consistent with the Braft evaluation. Details are provided in Appendix B.

### 7.1 Implementation

We implement CHIMERA-B by modifying Braft [8], which uses Raft as its underlying leader-based CFT consensus protocol. The implementation runs inside Intel TDX and integrates trusted metadata protection with protocol-guided log recovery.

For trusted metadata state, we adopt TIKS [80] and use Narrator-Pro [66] as the software-based trusted-counter backend, which advances a counter within one communication round. For sealing, we use the Intel SDK [38] to obtain the TDX measurement ( $mr\_td$ ) and derive a stable sealing key for protecting persistent state.

Realizing CHIMERA on Braft requires two main modifications:

- **Metadata.** Raft persists *currentTerm* and *votedFor* as election metadata (§5.1). Braft stores them in a single on-disk structure, *raftMeta*. CHIMERA associates each protected metadata transition with a trusted counter, enabling safe metadata recovery.
- **Log.** CHIMERA-B buffers newly appended log entries in memory and persists them asynchronously in the background, removing

synchronous disk writes from the commit critical path. During recovery, we reuse Braft’s existing *replicator* to transfer the required log suffix to recovering replicas.

**Baselines.** We implement four rollback-resilient baselines corresponding to the strategy families in §3:

- **Braft-TC** follows the trusted-counter approach used by En-raft [80], protecting state updates with a software-based counter. Each counter increment requires two rounds of broadcast.
- **Braft-RFT** follows FlexiBFT [31], increasing the replica set from  $2f + 1$  to  $3f + 1$  and using a trusted counter at the leader to prevent equivocation.
- **Braft-RC** uses Raft’s native reconfiguration mechanism based on joint consensus [62].
- **Braft-DCR** follows the rollback-resilient recovery strategy of Achilles [60], requiring a recovering replica to skip two epochs before rejoining. Braft’s Pre-Vote mechanism bounds epoch growth and makes this recovery rule applicable.

We additionally include **Braft-Direct Recovery (Braft-DR)** as a non-rollback-resilient reference point for quantifying protection overhead. Braft-DR directly restores sealed local state without any additional rollback-resilience mechanism.

## 7.2 Experimental Setup

We run all experiments on a public cloud platform using up to 61 Intel TDX-enabled instances, with one replica per virtual machine. Each instance has 4 vCPUs, 16 GB of memory, and runs a 64-bit Linux 5.10 LTS kernel.

We consider both LAN and WAN settings using Linux *tc* for traffic shaping. In the LAN setting, each replica has up to 10 Gbps bandwidth and inter-replica RTT remains below 1 ms. In the WAN setting, bandwidth is limited to 1 Gbps per replica, with a 40 ms RTT and  $\pm 4$  ms jitter. We use WAN emulation because TDX-enabled instances are currently restricted to a limited set of cloud regions.

**Parameters and Metrics.** We vary the fault-tolerance parameter as  $f \in \{1, 2, 4, 10, 20\}$ . The leader batches 256 client requests per log entry, each request carrying a 256 B payload. We bound the number of in-flight log entries at 5120 to control network usage.

We report *throughput*, *latency*, and *recovery time*. Recovery time is measured from replica restart until the replica completes CHIMERA recovery and is permitted to resume protocol participation.

**Evaluation Scope.** Our experiments quantify normal-path overhead, restart and recovery cost, recovery-time availability, and TEE-related overhead. The E1–E3 metadata crash windows are analyzed in §6.2 and covered by our bounded Maude model; the current implementation evaluation does not perform targeted fault injection at these individual boundaries.

## 7.3 Fault-Free Performance

This section evaluates CHIMERA-B’s log-commit performance without failures or leader changes. We compare it with Braft-TC, Braft-RFT, and Braft-DR. Braft-DCR and Braft-RC add no normal-case overhead beyond Braft-DR, so we omit their duplicate curves.

**7.3.1 Performance in WAN.** We evaluate throughput and latency under a WAN deployment while varying the fault threshold  $f$

(Fig. 3a and 3b). In this setting, communication overhead and bandwidth limits dominate performance.

Braft-TC incurs additional communication and trusted-counter operations on the commit path, while Braft-RFT increases quorum processing cost due to its larger replica set.

Both CHIMERA-B and Braft-DR operate with  $2f + 1$  nodes and incur no additional normal-case overhead, which explains their relatively higher throughput in this environment. Nevertheless, CHIMERA-B’s recovery mechanism does not rely on the completeness of on-disk logs, enabling it to safely defer disk writes to background operations and thus reduce normal-case commit latency. This design yields an average throughput improvement of approximately 10% over Braft-DR.

**7.3.2 Performance in LAN.** We also evaluate the throughput and latency of CHIMERA in a LAN deployment to minimize the effect of network communication (Fig. 3c and 3d). As the network communication cost is negligible in a LAN environment, local processing and I/O overhead become the dominant factors affecting performance.

CHIMERA-B exhibits a pronounced throughput advantage over all other variants due to its normal-case optimization that defers durable disk writes to background operations (as explained above). When  $f = 1$ , this optimization leads to a 68% improvement over Braft-DR that follows the original Braft I/O semantics of synchronous persistence.

Braft-TC incurs further penalties from its additional communication and trusted counter operations, though in the LAN setting, these penalties are partially masked by the low RTT. Braft-RFT, with its  $3f + 1$  configuration, experiences a more significant throughput drop as fault increases, due to the larger quorum size and corresponding processing and message handling overhead. Overall, CHIMERA-B shows superior performance when network delays are not the primary bottleneck and storage-layer optimizations directly translate into substantial end-to-end performance gains.

**7.3.3 Throughput vs. Latency.** Fig. 4 illustrates the latency of the CHIMERA-B and its counterparts with increasing throughput until system saturation in both LAN and WAN deployments. With 10 faulty nodes, CHIMERA reaches maximum throughputs of 135.8 kTPS in LAN and 22.6 kTPS in WAN, closely matching or surpassing the best performance among all counterparts. Notably, in LAN, CHIMERA-B even outperforms Braft-DR, as asynchronous disk writes reduce persistence bottlenecks. Overall, the results confirm that CHIMERA introduces negligible overhead to log consensus and can even enhance performance in certain settings.

## 7.4 Performance under Faults

Having evaluated fault-free performance, we next measure recovery overhead and its effect on availability. We compare CHIMERA-B with Braft-DR, Braft-TC, Braft-RC, and Braft-DCR; Braft-RFT is omitted because its recovery is identical to Braft-DR. In a 21-node LAN deployment with 210 MB/s sequential storage throughput and a preloaded 5 GB log per node, we shut down and restart 10 nodes while continuously issuing requests. After the shutdown, only  $f + 1$  replicas remain active, so another failure would prevent progress. **Single-Node Recovery Latency.** Table 3 divides the time for a recovering replica to rejoin the protocol into three stages.

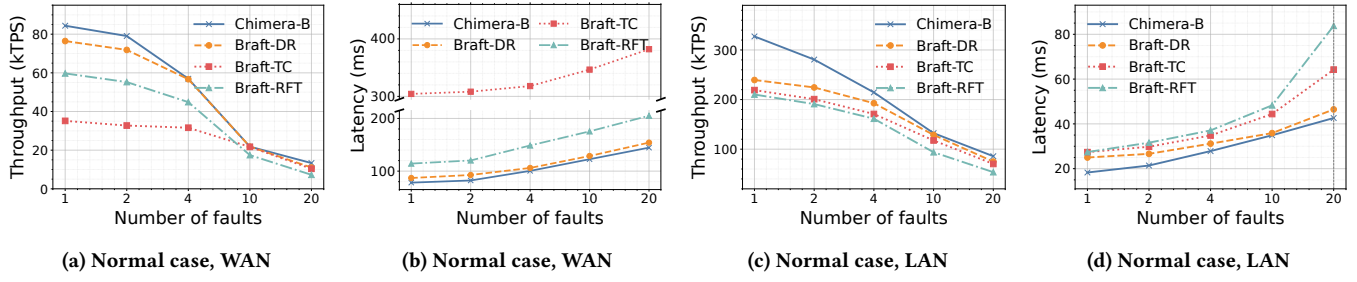


Figure 3: Throughput and latency comparisons with varying nodes in WAN and LAN.

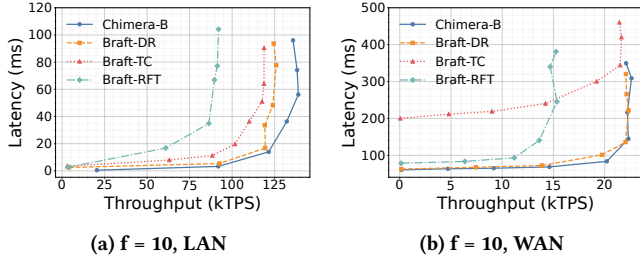


Figure 4: Throughput-latency under different workloads.

Table 3: Recovery overhead.

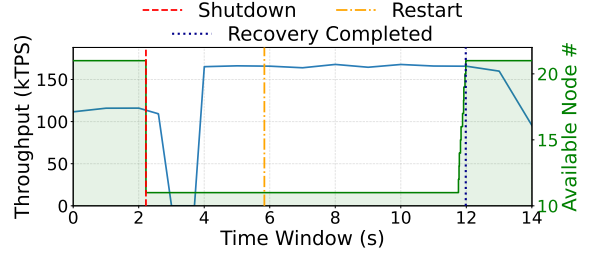
| Cost (s) | Braft-DR | Braft-TC | Braft-RC | Braft-DCR        | CHIMERA-B     |
|----------|----------|----------|----------|------------------|---------------|
| Prep.    | N/A      | 0.01     | 0.01     | N/A              | 0.01          |
| Sync.    | 6.15     | 6.31     | 117.01   | 24.53            | 6.42 (*18)    |
| Quie.    | N/A      | N/A      | N/A      | 2 epochs         | N/A           |
| Total    | 6.15     | 6.32     | 117.02   | 24.53 + 2 epochs | 6.43 (*18.01) |

- *Preparation.* Braft-TC and CHIMERA-B read the trusted counter in about 10 ms. Braft-RC performs two consensus rounds for reconfiguration, with a similar cost in the LAN. Its preparation latency therefore depends on inter-node message delays.
- *Synchronization.* Braft-RC reconstructs 5 GB of state from the leader in 117.01 s. Braft-DCR spends 24.53 s on disk loading and network catch-up. Optimized CHIMERA-B loads and validates its local log without network catch-up in 6.42 s; without this optimization, it takes approximately 18 s.
- *Quiescence.* Braft-DCR must additionally skip two epochs before rejoining, whereas the other mechanisms require no such delay.

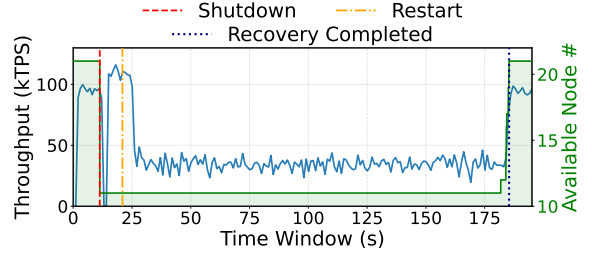
Overall, optimized CHIMERA-B rejoins in 6.43 s, close to Braft-DR’s 6.15 s, while avoiding Braft-RC’s full-state reconstruction and Braft-DCR’s synchronization and quiescence delays.

**Fault Recovery Process.** We next evaluate recovery process. Fig. 5a and Fig. 5b show the recovery process of CHIMERA-B and Braft-RC, while Table 3 summarizes recovery costs across all baselines.

Fig. 5a shows CHIMERA-B’s throughput during recovery. At the fault injection point (*i.e.*, 2.2 seconds), throughput briefly drops to zero while the leader handles failed connections. It then exceeds the pre-failure baseline because the same quorum is served through fewer connections, reducing communication overhead. This does not indicate higher availability: with only  $f + 1$  active replicas, any additional failure would prevent forming a quorum.



(a) CHIMERA-B



(b) Braft-RC

Figure 5: Fault recovery process of CHIMERA-B and Braft-RC.

After restart, the failed nodes reload their local logs with little effect on ongoing throughput and reach the re-entry point in about 6 s with the recovery optimization. Because the leader continues serving requests and extending the log during this interval, the recovered replicas may still need to fetch newly produced entries. This catch-up traffic temporarily reduces throughput before it returns toward the full-replica baseline. The recovered replicas fully resume replication after catch-up completes.

Fig. 5b shows the corresponding process for Braft-RC. Its shutdown behavior is similar, but reconfigured nodes synchronize their state from the leader rather than reuse valid local state. This synchronization traffic competes with request replication and reduces throughput until completion. The system can continue processing requests but cannot tolerate another replica failure. Braft-RC therefore remains with only  $f + 1$  active replicas longer, whereas CHIMERA-B restores the system’s failure margin sooner.

## 7.5 Overhead Profiling

To better understand the overhead of TEE-related operations, we compare variants of CHIMERA-B.

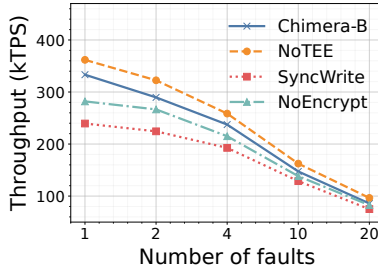


Figure 6: Overhead profiling of TEE-related execution.

- *NoTEE*. It runs outside Intel TDX, serving as a baseline to measure TEE-related overhead.
- *SyncWrite*. It uses synchronous disk writes instead of asynchronous persistence, isolating the cost of log persistence.
- *NoEncrypt*. It disables memory encryption atop *SyncWrite*, revealing the overhead of cryptographic operations.

Fig. 6 shows the throughput of CHIMERA-B and its variants as the number of faults ( $f$ ) increases in a LAN setting, with all parameters identical to the fault-free scenarios. Compared to *NoTEE*, CHIMERA-B incurs an 8–10% slowdown, capturing the inherent cost of TEE execution. Relative to *SyncWrite*, persistence adds roughly 15–20% overhead, independent of encryption. Finally, the difference between *SyncWrite* and *NoEncrypt* quantifies the cost of cryptographic sealing, adding 10–15% degradation. Overall, TEE-related overhead on the Braft path is modest.

## 8 RELATED WORK

**Confidential BFT Service.** Confidential BFT services have recently attracted significant attention from industry and academia, driven by the explosive growth of cloud and decentralized applications. Notable industrial examples include SVR3 [20] that ports Raft into TEEs to secure private key management, and Azure that provides Confidential Ledger service [56] atop CCF. Meanwhile, in academia, SecureKeeper [12] is among the first to use TEEs to protect metadata confidentiality in cloud settings with minimal changes to the Zab protocol [42]. Similarly, Brandenburger *et al.* [11] integrate Intel SGX into Hyperledger Fabric [2] to secure smart contract execution. CCF [34] uses enclaves to maintain a distributed key-value store and runs Raft [62] to achieve low latency and tolerate a minority of Byzantine faults.

However, most systems do not consider TEEs’ rollback attacks. Engraft [80] first identified this threat and introduced TIKS, *i.e.*, software-based counters, to enforce trusted counters for rollback protection. This approach corresponds to TC (as introduced in §3), which represents the most general solution. More details of the TC are introduced in Appendix C. Later versions of CCF [35] address the issue by reconfiguration, referred to as RC. However, these solutions degrade either performance or availability (§3).

**TEE-Assisted BFT Consensus.** Unlike confidential BFT consensus that ports whole consensus protocols into TEEs, TEE-Assisted BFT consensus [9, 21, 22, 31, 49, 60, 90] usually utilizes TEEs to provide some trusted functions, such as the append-only log and monotonic counter, to minimize Trusted Computing Base (TCB). These

trusted functions can prevent Byzantine nodes from equivocating messages, resulting in better scalability in terms of smaller quorum size and shorter transaction latency. Recently, FlexiBFT [31] and Achilles [60] identified TEEs’ rollback issues in TEE-Assisted BFT consensus and proposed RFT and DCR as solutions, respectively. However, RFT relaxes the tolerance, while DCR weakens the system’s availability. See more details in §3.

**Rollback-Resilient Storage.** Nimble [3] exposes an application API for replicating security-critical state, while Rollbaccine [18] transparently restores block-device crash consistency by interpreting persistence flags below the file system. HarborMaster [57] detects rollback attacks in TEE-based distributed clusters by offloading causality checking to a dedicated auditing service, distinguishing provability (evidence for detection) from retrievability (evidence for recovery). REBOUND [13] separates content rollback from continuous advancement of authorization and audit history, enabling policy-authorized rollbacks while preserving forward-committed state. These systems address rollback at application, storage, or audit boundaries, whereas CHIMERA targets consensus recovery in replicated data services. Its protocol-aware state mapping avoids synchronously protecting every log write and determines when a rolled-back replica may safely re-enter consensus. Thus, CHIMERA complements rather than replaces transparent storage defenses, crash consistency, or rollback detection mechanisms.

**Crash-Recovery and Sleepy Consensus.** Crash-recovery and sleepy-consensus protocols consider replicas that temporarily disappear and later rejoin [25, 52, 58]. The known-participation model is particularly relevant because it allows replicas without stable local state and reconstructs safety from other participants [79]. These approaches mainly address unavailable or stateless replicas. Our adversary can instead replay an earlier authenticated TEE state, allowing a replica to repeat a protocol action unless recovery restores non-equivocation. Their distributed reconstruction is closest to DCR in our taxonomy; CHIMERA instead protects node-local metadata with trusted counters, avoiding conservative epoch skipping while retaining the  $n = 2f + 1$  TEE-assisted resilience bound.

## 9 CONCLUSION

We systematize rollback-resilient recovery for confidential replicated services and show that uniform state protection creates a fundamental performance–availability trade-off. CHIMERA addresses this limitation by mapping node-local election metadata and replicated logs to distinct recovery mechanisms under an explicit leader-based consensus contract. We prove that CHIMERA preserves election safety and leader completeness and provides recovery liveness, and model check the Braft instantiation with Maude. We implement Braft/Raft and ZooKeeper/Zab prototypes on Intel TDX, using Braft for comprehensive evaluation and ZooKeeper/Zab as a portability case. Our evaluation demonstrates that CHIMERA introduces low normal-path overhead while significantly reducing recovery interruption compared with representative rollback-resilient baselines. By reusing valid local state and leveraging protocol-guided reconstruction, CHIMERA restores the system’s fault-tolerance margin sooner after failures and maintains the performance–availability balance across different deployment settings.

## REFERENCES

- [1] Gul A. Agha. 1990. *ACTORS - a model of concurrent computation in distributed systems*. MIT Press.
- [2] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolic, Sharon Weed Cocco, and Jason Yellick. 2018. Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains. In *Proc. of EuroSys*.
- [3] Sebastian Angel, Aditya Basu, Weidong Cui, Trent Jaeger, Stella Lau, Srinath Setty, and Sudheesh Singanamalla. 2023. Nimble: Rollback protection for confidential cloud services. In *Proc. of OSDI*.
- [4] Apache Software Foundation. [n.d.]. Apache ZooKeeper. <https://zookeeper.apache.org/>. Retrieved September 2025.
- [5] ARM. 2009. Security technology building a secure system using TrustZone technology (white paper).
- [6] Balaji Arun and Binoy Ravindran. 2022. Scalable Byzantine Fault Tolerance via Partial Decentralization. *Proc. VLDB Endow.* 15, 9 (2022), 1739–1752.
- [7] Microsoft Azure. [n.d.]. Confidential Consortium Framework. <https://www.microsoft.com/en-us/research/project/confidential-consortium-framework/>. Retrieved September 2025.
- [8] Baidu, Inc. [n.d.]. BRAFT. <https://github.com/baidu/braft>. Retrieved September 2025.
- [9] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. 2017. Hybrids on Steroids: SGX-Based High Performance BFT. In *Proc. of EuroSys*.
- [10] Alysson Bessani, Miguel Correia, Tobias Distler, Rüdiger Kapitza, Paulo Esteves-Verissimo, and Jiangshan Yu. 2023. Vivisection: The Dissection: On the Role of Trusted Components in BFT Protocols. *arXiv preprint arXiv:2312.05714* (2023).
- [11] Marcus Brandenburger, Christian Cachin, Rüdiger Kapitza, and Alessandro Sorniotti. 2019. Trusted Computing Meets Blockchain: Rollback Attacks and a Solution for Hyperledger Fabric. In *Proc. of SRDS*.
- [12] Stefan Brenner, Colin Wulf, David Goltzsche, Nico Weichbrodt, Matthias Lorenz, Christof Fetzer, Peter Pietzuch, and Rüdiger Kapitza. 2016. SecureKeeper: Confidential ZooKeeper using Intel SGX. In *Proc. of Middleware*.
- [13] Quinn Burke, Anjo Vahldiek-Oberwagner, Michael Swift, and Patrick McDaniel. 2026. It's a Feature, Not a Bug: Secure and Auditable State Rollback for Confidential Cloud Applications. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE. arXiv:2511.13641
- [14] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine fault tolerance. In *Proc. of OSDI*.
- [15] Tushar D Chandra, Robert Griesemer, and Joshua Redstone. 2007. Paxos made live: An engineering perspective. In *Proc. of PODC*.
- [16] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H. Lai. 2019. SgxPectre: Stealing Intel Secrets from SGX Enclaves Via Speculative Execution. In *Proc. of EuroS&P*.
- [17] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah Johnson, Ari Juels, Andrew Miller, and Dawn Song. 2019. Ekiden: A Platform for Confidentiality-Preserving, Trustworthy, and Performant Smart Contracts. In *Proc. of EuroS&P*.
- [18] David C. Y. Chu, Aditya Balasubramanian, Dee Bao, Natacha Crooks, Heidi Howard, Lucky Katahanas, and Soujanya Ponnappalli. 2026. Rollbaccine: Herd Immunity against Storage Rollback Attacks in TEEs. *Proc. ACM Manag. Data* 4, 1 (2026), 79:1–79:27.
- [19] Allen Clement, Flavio Junqueira, Aniket Kate, and Rodrigo Rodrigues. 2012. On the (limited) power of non-equivocation. In *Proc. of PODC*.
- [20] Graeme Connell, Vivian Fang, Rolfe Schmidt, Emma Dauterman, and Raluca Ada Popa. 2024. Secret Key Recovery in a Global-Scale End-to-End Encryption System. In *Proc. of OSDI*.
- [21] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. 2022. DAMYSUS: Streamlined BFT Consensus Leveraging Trusted Components. In *Proc. of EuroSys*.
- [22] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. 2024. OneShot: View-Adapting Streamlined BFT Protocols with Trusted Execution Environments. In *Proc. of IPDPS*.
- [23] Baltasar Dinis, Peter Druschel, and Rodrigo Rodrigues. 2023. RR: A Fault Model for Efficient TEE Replication. In *Proc. of NDSS*.
- [24] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (1988), 288–323.
- [25] Yuval Efron, Joachim Neu, Ling Ren, and Ertem Nusret Tas. 2026. Optimal Good-Case Latency for Sleepy Consensus. In *Proc. of EUROCRYPT*. 397–426.
- [26] Hang Feng, Yufeng Hu, Yinghan Kou, Runhui Li, Jianfeng Zhu, Lei Wu, and Yajin Zhou. 2024. SlimArchive: A Lightweight Architecture for Ethereum Archive Nodes. In *Proc. of USENIX ATC*.
- [27] Fangyu Gai, Jianyu Niu, Ivan Beschastnikh, Chen Feng, and Sheng Wang. 2023. Scaling blockchain consensus via a robust shared mempool. In *Proc. of ICDE*.
- [28] Shabnam Ghasemirad, Si Liu, Christoph Sprenger, Luca Multazzu, and David Basin. 2025. VerIso: Verifiable Isolation Guarantees for Database Transactions. *Proc. VLDB Endow.* 18, 5 (Aug. 2025), 1362–1375.
- [29] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. 2003. The Google file system. In *Proc. of SOSP*.
- [30] Dimitra Giantsidi, Emmanouil Giortamis, Julian Pritzi, Maurice Bailleu, Manos Kapritsos, and Pramod Bhatotia. 2025. Recipe: Hardware-Accelerated Replication Protocols. *arXiv preprint* (2025). arXiv:2502.09251
- [31] Suyash Gupta, Sajjad Rahnama, Shubham Pandey, Natacha Crooks, and Mohammad Sadoghi. 2023. Dissecting BFT Consensus: In Trusted Components We Trust!. In *Proc. of EuroSys*.
- [32] James Hamilton. 2007. On designing and deploying internet-scale services. In *Proc. of LISA*.
- [33] Matthew Hoekstra, Reshma Lal, Pradeep Pappachan, Vinay Phegade, and Juan Del Cuvillo. 2013. Using Innovative Instructions to Create Trustworthy Software Solutions. In *Proc. of HASP*.
- [34] Heidi Howard, Fritz Alder, Edward Ashton, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Antoine Delignat-Lavaud, Cédric Fournet, Andrew Jeffery, Matthew Kerner, Fotios Kounelis, Markus A. Kuppe, Julien Maffre, Mark Russinovich, and Christoph M. Wintersteiger. 2023. Confidential Consortium Framework: Secure Multiparty Applications with Confidentiality, Integrity, and High Availability. *Proc. VLDB Endow.* 17, 2 (2023), 225–240.
- [35] Heidi Howard, Markus A. Kuppe, Edward Ashton, Amaury Chamayou, and Natacha Crooks. 2025. Smart Casual Verification of the Confidential Consortium Framework. In *Proc. of NSDI*.
- [36] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. 2010. ZooKeeper: Wait-Free Coordination for Internet-Scale Systems. In *Proc. of ATC*.
- [37] Intel Corporation. [n.d.]. Intel Trust Domain Extensions. <https://www.intel.com/content/dam/develop/external/us/en/documents/tdx-whitepaper-final9-17.pdf>. Retrieved September 2025.
- [38] Intel Corporation. [n.d.]. SGX Data Center Attestation Primitives. <https://github.com/intel/SGXDataCenterAttestationPrimitives>. Retrieved September 2025.
- [39] Intel Corporation. [n.d.]. Trusted Time and Monotonic Counters with Intel Software Guard Extensions Platform Services. <https://www.intel.com/content/www/us/en/content-details/671564/trusted-time-and-monotonic-counters-with-intel-software-guard-extensions-platform-services.html>. Retrieved September 2025.
- [40] Mohammad M. Jalalzai, Jianyu Niu, Chen Feng, and Fangyu Gai. 2024. Fast-HotStuff: A Fast and Robust BFT Protocol for Blockchains. *IEEE Trans. Dependable Secure Comput.* 21, 4 (2024), 2478–2493.
- [41] Andrew Jeffery, Julien Maffre, Heidi Howard, and Richard Mortier. 2024. LSKV: A Confidential Distributed Datastore to Protect Critical Data in the Cloud. *arXiv preprint arXiv:2407.12623* (2024).
- [42] Flavio P. Junqueira, Benjamin C. Reed, and Marco Serafini. 2011. Zab: High-performance broadcast for primary-backup systems. In *Proc. of DSN*.
- [43] David Kaplan, Jeremy Powell, and Tom Woller. 2020. AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. Technical Report. AMD. <https://www.amd.com/system/files/TechDocs/SEV-SNP-strengthening-vm-isolation-with-integrity-protection.pdf>
- [44] Gabriel Kapchuk, Ian Miers, and Matthew Green. 2019. Giving state to the stateless: Augmenting trustworthy computation with ledgers. In *Proc. of NDSS*.
- [45] Jae-Yun Kim, Junmo Lee, Yeonjae Koo, Sanghyeon Park, and Soo-Mook Moon. 2021. Ethanos: efficient bootstrapping for full nodes on account-based blockchain. In *Proc. of EuroSys*.
- [46] Jan Kończak, Nuno Santos, Tomasz Żurkowski, Paweł T. Wojciechowski, and André Schiper. 2011. *JPaxos: State Machine Replication Based on the Paxos Protocol*. Technical Report 167765. EPFL. <https://infoscience.epfl.ch/handle/20.500.14299/69874>
- [47] Joshua Lind, Oded Naor, Ittay Eyal, Florian Kelbert, Emin Gün Sirer, and Peter Pietzuch. 2019. Teechain: A Secure Payment Network with Asynchronous Blockchain Access. In *Proc. of SOSP*.
- [48] Barbara Liskov and James Cowling. 2012. *Viewstamped Replication Revisited*. Technical Report MIT-CSAIL-TR-2012-021. MIT CSAIL.
- [49] Jian Liu, Wenting Li, Ghassan O. Karame, and N. Asokan. 2019. Scalable Byzantine Consensus via Hardware-Assisted Secret Sharing. *IEEE Trans. Comput.* 68 (2019), 139–151.
- [50] Si Liu, Peter Csaba Ölveczky, Min Zhang, Qi Wang, and José Meseguer. 2019. Automatic Analysis of Consistency Properties of Distributed Transaction Systems in Maude. In *TACAS 2019 (LNCS)*, Vol. 11428. Springer, 40–57.
- [51] Si Liu, Muntasir Raihan Rahman, Stephen Skeirik, Indranil Gupta, and José Meseguer. 2014. Formal Modeling and Analysis of Cassandra in Maude. In *ICFEM 2014 (LNCS)*, Vol. 8829. Springer, 332–347.
- [52] Dahlia Malkhi, Atsuki Momose, and Ling Ren. 2023. Towards Practical Sleepy BFT. In *Proc. of CCS*. 490–503.
- [53] André Martin, Cong Lian, Franz Gregor, Robert Krahn, Valerio Schiavoni, Pascal Felber, and Christof Fetzer. 2021. ADAM-CS: Advanced Asynchronous Monotonic Counter Service. In *Proc. of DSN*.

- [54] Sinisa Matetic, Mansoor Ahmed, Kari Kostiaainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. 2017. ROTE: Rollback Protection for Trusted Execution. In *Proc. of USENIX Security*.
- [55] Ellis Michael, Dan R. K. Ports, Naveen Kr. Sharma, and Adriana Szekeres. 2016. *Providing Stable Storage for the Diskless Crash-Recovery Failure Model*. Technical Report UW-CSE-16-08-02. University of Washington.
- [56] Microsoft Azure. [n.d.]. Microsoft Azure Confidential Ledger. <https://learn.microsoft.com/en-us/azure/confidential-ledger/overview>. Retrieved September 2025.
- [57] Shubham Mishra, Alexander Thomas, Nurzhan Abdrassilov, Kaiyuan Chen, Nat-acha Crooks, and John Kubiawicz. 2026. HarborMaster: Rollback Detection for Trusted Distributed Computing. *Proceedings of the VLDB Endowment* 19, 9 (2026), 2126–2139.
- [58] Atsuki Momose and Ling Ren. 2022. Constant Latency in Sleepy Consensus. In *Proc. of CCS*. 2295–2308.
- [59] Jianyu Niu, Wei Peng, Xiaokuan Zhang, and Yinqian Zhang. 2022. NARRATOR: Secure and Practical State Continuity for Trusted Execution in the Cloud. In *Proc. of CCS*.
- [60] Jianyu Niu, Xiaoqing Wen, Guanlong Wu, Shengqi Liu, Jiangshan Yu, and Yinqian Zhang. 2025. Achilles: Efficient TEE-Assisted BFT Consensus via Rollback Resilient Recovery. In *Proc. of EuroSys*.
- [61] Oleksii Oleksenko, Bohdan Trach, Robert Krahn, Mark Silberstein, and Christof Fetzer. 2018. Varys: Protecting SGX Enclaves from Practical Side-Channel Attacks. In *Proc. of USENIX ATC*.
- [62] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proc. of ATC*.
- [63] Alina Oprea and Michael K Reiter. 2007. Integrity Checking in Cryptographic File Systems with Constant Trusted Storage. In *Proc. of USENIX Security*.
- [64] Lingzhi Ouyang, Xudong Sun, Ruize Tang, Yu Huang, Madhav Jivrajani, Xiaoxing Ma, and Tianyin Xu. 2025. Multi-Grained Specifications for Distributed System Model Checking and Verification. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 379–395.
- [65] Bryan Parno, Jacob R. Lorch, John R. Douceur, James Mickens, and Jonathan M. McCune. 2011. Memoir: Practical State Continuity for Protected Modules. In *Proc. of S&P*.
- [66] Wei Peng, Xiang Li, Jianyu Niu, Xiaokuan Zhang, and Yinqian Zhang. 2024. Ensuring State Continuity for Confidential Computing: A Blockchain-Based Approach. *IEEE Trans. Dependable Secure Comput.* 21, 6 (2024), 5635–5649.
- [67] Christian Priebe, Kapil Vaswani, and Manuel Costa. 2018. EnclaveDB: A secure database using SGX. In *Proc. of S&P*. IEEE.
- [68] Mark Russinovich, Edward Ashton, Christine Avanesians, Miguel Castro, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Cédric Fournet, Matthew Kerner, Sid Krishna, et al. 2019. *CCF: A Framework for Building Confidential Verifiable Replicated Services*. Technical Report. Microsoft Research and Microsoft Azure.
- [69] Luis F. G. Sarmenta, Marten van Dijk, Charles W. O'Donnell, Jonathan Rhodes, and Srinivas Devadas. 2006. Virtual Monotonic Counters and Count-Limited Objects Using a TPM without a Trusted OS. In *Proc. of STC*.
- [70] Michael D. Schroeder, Andrew D. Birrell, and Roger M. Needham. 1984. Experience with Grapevine: the growth of a distributed system. *ACM Trans. Comput. Syst.* 2, 1 (1984), 3–23.
- [71] Judah Schvimer, A. Jesse Jiryu Davis, and Max Hirschhorn. 2020. eXtreme Modelling in Practice. *Proc. VLDB Endow.* 13, 9 (2020), 1346–1358.
- [72] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. 2019. ZombieLoad: Cross-Privilege-Boundary Data Sampling. In *Proc. of CCS*.
- [73] Ming-Wei Shih, Sangho Lee, Taesoo Kim, and Marcus Peinado. 2017. T-SGX: Eradicating Controlled-Channel Attacks Against Enclave Programs. In *Proc. of NDSS*.
- [74] Signal Messenger LLC. [n.d.]. Signal Secure Value Recovery. <https://signal.org/blog/secure-value-recovery>. Retrieved September 2025.
- [75] Raoul Strackx, Bart Jacobs, and Frank Piessens. 2014. ICE: A Passive, High-Speed, State-Continuity Scheme. In *Proc. of ACSAC*.
- [76] Raoul Strackx and Frank Piessens. 2016. Ariadne: A Minimal Approach to State Continuity. In *Proc. of USENIX Security*.
- [77] Marten van Dijk, Jonathan Rhodes, Luis F. G. Sarmenta, and Srinivas Devadas. 2007. Offline Untrusted Storage with Immediate Detection of Forking and Replay Attacks. In *Proc. of STC*.
- [78] Stephan Van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue in-flight data load. In *Proc. of S&P*. IEEE.
- [79] Chenxu Wang, Sisi Duan, Minghui Xu, Feng Li, and Xiuzhen Cheng. 2026. Consensus in the Known Participation Model with Byzantine Faults and Sleepy Replicas. In *Proc. of NDSS*.
- [80] Weili Wang, Sen Deng, Jianyu Niu, Michael K Reiter, and Yinqian Zhang. 2022. Engraft: Enclave-guarded raft on Byzantine faulty nodes. In *Proc. of CCS*.
- [81] Weili Wang, Jianyu Niu, Michael K Reiter, and Yinqian Zhang. 2024. Formally Verifying a Rollback-Prevention Protocol for TEEs. In *Proc. of FORTE*.
- [82] Xiaoqing Wen, Quanbi Feng, Hanzheng Lyu, Jianyu Niu, Yinqian Zhang, and Chen Feng. 2025. TeeRollup: Efficient Rollup Design Using Heterogeneous TEE. In *IEEE Transactions on Computers*.
- [83] Xiaoqing Wen, Quanbi Feng, Jianyu Niu, Yinqian Zhang, and Chen Feng. 2026. Mercury: Practical Cross-Chain Exchange via Trusted Hardware. *IEEE Transactions on Dependable and Secure Computing* 23, 2 (2026), 2949–2961.
- [84] Annika Wilde, Tim Niklas Gruel, Claudio Soriente, and Ghassan Karame. 2024. The Forking Way: When TEEs Meet Consensus. *arXiv preprint* (2024). arXiv:2412.00706
- [85] Shaokang Xie, Dakai Kang, Hanzheng Lyu, Jianyu Niu, and Mohammad Sadoghi. 2025. Fides: Scalable Censorship-Resistant DAG Consensus via Trusted Components. (2025). arXiv:2501.01062
- [86] Hiroyuki Yamada and Jun Nemoto. 2022. Scalar DL: Scalable and Practical Byzantine Fault Detection for Transactional Database Systems. *Proc. VLDB Endow.* 15, 7 (2022), 1324–1336.
- [87] Ying Yan, Changzheng Wei, Xuepeng Guo, Xuming Lu, Xiaofu Zheng, Qi Liu, Chenhui Zhou, Xuyang Song, Boran Zhao, Hui Zhang, et al. 2020. Confidentiality Support over Financial Grade Consortium Blockchain. In *Proc. of ACM SIGMOD*.
- [88] Xinying Yang, Cong Yue, Wenhui Zhang, Yang Liu, Beng Chin Ooi, and Jianjun Chen. 2024. SecuDB: An In-enclave Privacy-preserving and Tamper-resistant Relational Database. *Proc. VLDB Endow.* 17, 12 (2024), 3906–3919.
- [89] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proc. of PODC*.
- [90] Jiashuo Zhang, Jianbo Gao, Ke Wang, Zhenhao Wu, Yue Li, Zhi Guan, and Zhong Chen. 2022. TBFT: Efficient Byzantine Fault Tolerance Using Trusted Execution Environment. In *Proc. of ICC*.

## A COMPLETE CORRECTNESS ARGUMENT

This appendix expands the proof sketch in the main paper. The argument is compositional: we assume the protected base protocol is correct during normal execution and prove that CHIMERA preserves the state needed by that protocol across crashes and rollback.

### A.1 Definitions and Assumptions

There are  $n = 2f + 1$  statically configured replicas. A *quorum* is any set of at least  $f + 1$  distinct replicas. A replica is *eligible* when its TEE has completed metadata and log recovery. A non-eligible replica cannot vote, propose, acknowledge a log entry, or answer another replica’s recovery request. Consequently, a signed recovery response certifies that the sender crossed the re-entry barrier, although a corrupted host may still suppress that response.

A *support action* for replica  $p_j$  in epoch  $e$  is either an ordinary election vote for  $p_j$  or a `REPLYRECOVER` response to  $p_j$ ’s Recovery Election. A *leadership certificate* is a quorum of support actions for one replica in one epoch. Duplicate transmissions of the same persisted support record are one action.

For logs,  $L_1 \preceq_s L_2$  denotes the base protocol’s safe-log relation. It is stronger than comparing lengths: if an eligible log contains a committed prefix  $P$  and the protocol selects a log that dominates it, the selected log also contains  $P$ . Raft realizes this obligation through its log-matching and election restrictions; Zab realizes it through discovery and synchronization. CHIMERA uses the relation exposed by the adapter and does not claim that one universal comparison works for every protocol.

The proof relies on four groups of assumptions.

- **A1 (Trusted persistence).** Each trusted counter is bound to one replica identity, is initialized correctly, and cannot decrease or be forked across crashes. Sealing provides authenticity but not freshness.
- **A2 (Protected execution and identity).** TEE code executes as specified; replica keys remain secret; accepted protocol and recovery messages are TEE-authenticated and nonce-bound; and at most one live TEE instance uses a replica identity.
- **A3 (Base-protocol contract).** The base protocol has monotonic epochs, majority leadership certificates, prefix-preserving log repair, append-only leaders, and a safe-log relation  $\preceq_s$ . An eligible replica never removes a committed prefix. Entry into an epoch closes the replica’s one support opportunity before the action becomes externally visible.
- **A4 (Faults, network, and membership).** Membership is static during recovery. At most  $f$  replicas are corrupted, crashed, or recovering at any time; a recovering replica remains in this set until it is eligible. After GST, messages among uncorrupted replicas arrive within  $\Delta$ , local TEE operations terminate, and the base protocol’s pacemaker eventually establishes a reachable leader.

### A.2 Recovery Invariants

The design maintains the following invariants. They are stated here rather than in the main paper because they support the complete proof rather than the high-level protocol description.

- **I1 (Counter-epoch binding).** After a completed protected transition, sealed metadata contains the counter value returned for

that transition and the same epoch. A multi-epoch jump executes one transition per epoch.

- **I2 (Monotonic repair).** A matching sealed counter restores the authentic support record. On a mismatch, recovery adopts the counter’s current value as its epoch and disables support in that epoch.
- **I3 (One support action).** Before an ordinary vote or recovery vote is sent, the supported identity is sealed. A replica that has entered an epoch may only retransmit support for that identity; it cannot create support for a different identity in the same epoch.
- **I4 (Role-directed fresh epoch).**  $TC_{role}$  determines whether a node crashed as follower or leader. Only a former leader starts a Recovery Election, and it completes a protected transition to a strictly larger epoch before sending `LEADERRECOVER`.
- **I5 (Shared-vote Recovery Election).** An eligible replica answers `LEADERRECOVER` only while entering the request’s epoch. The binding update records the recovering leader as its supported identity, so the response and an ordinary election vote consume the same support budget.
- **I6 (Safe log and re-entry).** A recovering leader proposes only after collecting a recovery quorum and selecting a log using  $\preceq_s$ . A recovering follower validates the predecessor tag and repairs any conflicting suffix. Neither crosses the re-entry barrier before holding the committed prefix required by the base protocol.
- **I7 (Election fallback).** If Recovery Election does not obtain a quorum because an ordinary election advances or consumes the target epoch, the recovering node does not propose. On observing an established leader in an equal or later epoch, it changes to follower and uses follower recovery.

I1–I3 follow from the binding update and its E1–E3 crash handling. I4–I5 follow from  $TC_{role}$  and the leader-recovery handler. I6 is enforced by the recovery quorum, the base-protocol adapter, and the re-entry barrier. I7 follows from the shared support budget and the base pacemaker.

### A.3 Supporting Lemmas

**LEMMA 1 (RECOVERED EPOCH MONOTONICITY).** *For any replica, the epoch obtained after recovery is no smaller than the highest epoch it entered before the crash.*

**PROOF.** Let  $e$  be the last epoch entered before the crash. By I1, entering  $e$  advanced  $TC_{md}$  to at least  $e$  and sealed that counter with the metadata. By A1, the value  $v = \text{ReadTC}(md)$  at recovery cannot be smaller. If sealed metadata matches  $v$ , authenticity and I1 restore its epoch. If it does not match, I2 sets the epoch to  $v$ . Both cases recover an epoch at least  $e$ .  $\square$

**LEMMA 2 (SUPPORT UNIQUENESS).** *A replica produces support actions for at most one identity in any epoch.*

**PROOF.** Without a crash, I3 persists one identity before exposing an action and rejects any different same-epoch request. After a crash, matching metadata restores that identity, so only the same action can be retransmitted. A mismatch invokes I2 and disables support. By Lemma 1, recovery cannot return to an earlier epoch

to create another action there. I5 applies these same rules to REPLYRECOVER; hence ordinary and recovery votes are not separate opportunities.  $\square$

LEMMA 3 (ELECTION-PATH EXCLUSION). *An ordinary election and a Recovery Election cannot both establish different leaders in the same epoch.*

PROOF. Let  $Q_o$  and  $Q_r$  be their leadership certificates. Both contain at least  $f + 1$  identities, so  $Q_o \cap Q_r \neq \emptyset$  because  $n = 2f + 1$ . A replica in the intersection would have supported two identities in one epoch, contradicting Lemma 2.  $\square$

LEMMA 4 (POST-RECOVERY TAG FRESHNESS). *After a leader crash, no new proposal from that recovered identity reuses an epoch-index tag that it used before the crash for different contents.*

PROOF. By I4 and Lemma 1, the recovered leader proposes only in a strictly later epoch. Even if it lost an uncommitted entry at index  $k$  from epoch  $e$ , its next entry at  $k$  is tagged  $(e', k)$  with  $e' > e$ . Repeated crashes repeat the strict advance. Thus the ordinary (epoch, index) tag is sufficient; no persistent restart identifier is needed.  $\square$

LEMMA 5 (COMMITTED-PREFIX PRESERVATION). *If prefix  $P$  is committed before a leadership transition, the leader established by either an ordinary election or Recovery Election contains  $P$  before proposing.*

PROOF. Let  $C$  be a commit quorum that acknowledged the last entry of  $P$ , and let  $Q$  be the later election or recovery quorum. Majority intersection gives a replica  $r \in C \cap Q$ . Membership in  $Q$  means that  $r$  is eligible when it supports the leader. If  $r$  never restarted, A3 prevents it from removing  $P$ . If it restarted, I6 prevented re-entry until it reconstructed  $P$ . Therefore  $r$ 's response carries  $P$ . The base protocol's safe-log relation selects a log that preserves every committed prefix represented in the quorum, so the new leader contains  $P$  before proposing.  $\square$

## A.4 The Three Recovery Properties

THEOREM 1 (ELECTION SAFETY). *At most one replica acts as leader in any epoch.*

PROOF. Every acting leader has a leadership certificate, whether obtained through an ordinary election or Recovery Election. Any two such certificates intersect. Their common replica cannot support different identities by Lemma 2; therefore two distinct leaders cannot both obtain certificates. Lemma 4 additionally ensures that a recovered leader does not recreate old-epoch proposal authority under an ambiguous entry tag.  $\square$

THEOREM 2 (LEADER COMPLETENESS). *If an entry is committed at index  $k$  in epoch  $e$ , every leader established in an epoch  $e' > e$  contains that entry at index  $k$  before proposing.*

PROOF. Let  $P$  be the committed prefix ending at that entry. Apply induction over leadership transitions after epoch  $e$ . For the first transition, Lemma 5 places  $P$  in the new leader's log. A3 makes the leader append-only, so it retains  $P$  throughout its tenure. For the induction step, assume all preceding leaders retained  $P$ . The

next leader again obtains either an ordinary or recovery quorum, and Lemma 5 places  $P$  in its selected log. Thus every later leader contains the committed entry at the same index.  $\square$

THEOREM 3 (RECOVERY LIVENESS). *If an uncorrupted replica starts recovery and remains uncorrupted, it eventually becomes eligible after GST.*

PROOF. Metadata recovery invokes a finite sequence of local counter and sealing operations, which terminate by A4. If the role counter indicates follower, the replica repeatedly requests recovery. The base pacemaker eventually provides a reachable eligible leader after GST; authenticated suffix transfer and validation then establish I6.

If the role counter indicates leader, the replica starts Recovery Election in a fresh epoch. If it obtains  $f + 1$  responses, it selects and synchronizes a safe log and becomes eligible. Otherwise, votes may be withheld or an ordinary election may have consumed the epoch. The base pacemaker eventually establishes a leader in an equal or later epoch; I7 then switches the replica to follower recovery, reducing to the preceding case. Network delivery and local processing are bounded after GST, so one of these paths eventually crosses the re-entry barrier.  $\square$

## B PORTABILITY TO ZOOKEEPER

We implement CHIMERA-Z on ZooKeeper [4], which uses Zab [42], to assess portability beyond Raft-specific election state. This case study examines how CHIMERA's separation of protected control state and replicated history maps to a protocol with different leader-establishment semantics.

### B.1 Mapping the Recovery Contract

Raft records its per-term voting decision explicitly in *votedFor*. Zab instead maintains persistent epoch state associated with discovery and synchronization. Fast Leader Election (FLE) nominates a prospective leader, which must complete these phases before entering normal broadcast [42].

**Epoch acceptance.** During discovery, the prospective leader proposes an epoch  $ep^*$  greater than the accepted epochs reported by a quorum. A follower that accepts this proposal persists  $ep^*$  as *acceptedEpoch* before acknowledging it. This field records the latest epoch proposal accepted by the follower.

**History synchronization.** During synchronization, followers reconcile their histories with the leader's selected history. Upon accepting NEWLEADER for the new epoch, a follower records *currentEpoch* before sending the corresponding acknowledgment. The leader enters broadcast only after receiving a quorum of new-leader acknowledgments [42].

The two fields therefore record distinct stages of leader establishment: accepting an epoch proposal and accepting the leader and synchronized history for that epoch. Their monotonicity supports Zab's safety together with its quorum and history-selection rules; the fields alone do not establish leadership. For CHIMERA, they represent protocol-specific control obligations rather than a literal replacement for Raft's *votedFor* field.

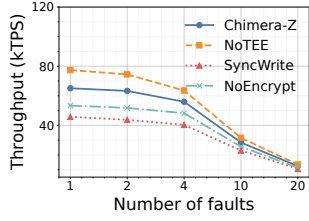


Figure 7: Overhead profiling of TEE-related execution.

## B.2 Implementation atop ZooKeeper

We run CHIMERA-Z inside Intel TDX. The port follows the same state-separation principle as CHIMERA-B, with the main adaptation in the protection of election metadata.

**Metadata protection.** Because *acceptedEpoch* and *currentEpoch* advance at different protocol steps, we protect them with separate trusted counters rather than combining them into a single Raft-style metadata record. Each counter is associated with its corresponding epoch field and sealed state. Integrating this protection must preserve Zab’s update-before-acknowledgment ordering: a safety-critical metadata transition must be protected before its acknowledgment becomes externally visible.

Separate counters do not make the two fields semantically independent. Recovery must preserve their ordering and their relationship with the synchronized history. In particular, validating the freshness of an epoch field does not, by itself, authorize the replica to resume normal protocol participation.

**Log persistence and recovery.** Log-persistence changes follow the approach used in CHIMERA-B: new entries are buffered in memory and persisted in the background. The log-recovery integration follows Zab’s history-synchronization semantics rather than directly applying Raft’s voting or log-selection rules. Thus, the portable design principle is to protect local control obligations separately from replicated history; the protocol-specific adaptation identifies the relevant metadata transitions and the conditions under which a recovered history permits participation.

## B.3 Evaluation

Fig. 7 presents the maximum throughput of CHIMERA-Z and its variants under the same overhead-profiling methodology as §7.5. The performance gaps are smaller than on Braft because ZooKeeper’s additional components and coordination limit peak throughput, masking much of the relative impact of TEE execution and persistence. The implementation required only targeted changes to log persistence and a two-counter metadata scheme for Zab’s two-phase epoch fields (§7.1), confirming that the protocol-aware recovery design transfers to Paxos-family protocols without fundamental redesign. CHIMERA-Z therefore answers Q4 affirmatively.

## C TRUSTED COUNTER DETAILS

A trusted counter is the most representative and common method for rollback prevention. Generally, there are two types of counters: hardware-based and software-based. The first includes SGX counter [39], TPM counter [53], and TPM NVRAM [65, 75, 76]. Hardware-based counters usually have poor performance, *i.e.*, long

latency (*e.g.*, tens of milliseconds) for read and write operations and limited write cycles [54, 59].

The second is virtual counters, which can be implemented by a single-write multiple-read register [59, 66, 81] or an append-only ledger [3, 44]. The former includes ROTE [54] and Narrator [59], which adopt a two-phase broadcast protocol. The latter can be realized by blockchain [44] or CFT consensus as adopted in Nimble [3]. However, using them in confidential BFT consensus protocols introduces several communication steps. In this paper, we use Narrator-Pro, *i.e.*, a single-write multiple-read register [66], as the software-based counters. Despite their high costs, we employ counters only for infrequently updated metadata, thereby avoiding protocol overhead for log commitment.

**Atomicity dilemma.** There are two fundamental operations when using a trusted counter: incrementing the counter and sealing the state. Because these operations cannot be executed atomically, their sequence leads to two distinct patterns.

- *Inc-then-store pattern* [54, 59, 66, 80]. In this approach, the counter is incremented before the state is sealed. This guarantees that no previously sealed state can be replayed, since the counter always moves forward. However, if a crash occurs after the counter has been incremented but before the state is sealed, the counter and state become permanently inconsistent, making recovery impossible. From the recovering node’s perspective, a mismatch between the counter values is indistinguishable from either a benign crash or a deliberate rollback attack.
- *Store-then-inc pattern* [65, 76]. The state is sealed first, and then the counter is incremented. This avoids unrecoverable crashes, since the sealed state always exists even if the counter increment fails. However, this introduces a rollback window: an adversary can seal multiple different states under the same counter value and later replay an old one. After a reboot, the node cannot determine which state with the same counter value is most recent, allowing its state to be rolled back.

## D FORMAL MODELING AND ANALYSIS

Our formal specification of CHIMERA-B consists of approximately 910 LoC in Maude, source code is available at <https://github.com/Artifacts2026/CHIMERA/MaudeSpec>. Our modeling follows Agha’s *actors* paradigm [1]. Specifically, nodes are modeled as actors, and their communication is captured through message passing. Upon receiving a message, a node may update its local state and possibly generate new messages. The overall system evolves through such message-triggered transitions.

We verify CHIMERA-B under TEE rollbacks using linear-temporal logic (LTL) model checking, focusing on four key properties:

- **P1. Election Safety:** at most one leader can be elected in any epoch.
- **P2. Leader Completeness:** once a log entry is committed by a leader, any subsequent leader contains this entry.
- **P3. Recovery Liveness:** a node undergoing reboot eventually completes its recovery procedure.
- **P4. Protocol Liveness:** every client request is eventually committed as a log entry.

Table 4 shows the model checking results for three cases: three nodes with one reboot, three nodes with two reboots, and five

**Table 4: Model checking results.**

| Property | Metric  | 3 Nodes | 3 Nodes w/ 2 reboots | 5 Nodes        |
|----------|---------|---------|----------------------|----------------|
| P1       | #States | /       | 2,313,191 [45]       | 2,200,414 [26] |
|          | Time    | 557s    | 1,174s               | 4,209s         |
| P2       | #States | /       | 2,691,520 [45]       | 2,652,353 [26] |
|          | Time    | 1,294s  | 2,883s               | 9,814s         |
| P3       | #States | /       | 2,691,520 [45]       | 2,652,353 [26] |
|          | Time    | 1,145s  | 3,274s               | 10,213s        |
| P4       | #States | /       | 2,691,520 [45]       | 2,652,353 [26] |
|          | Time    | 1,146s  | 3,369s               | 10,469s        |

nodes. Note that the state space grows rapidly with additional nodes, making exhaustive verification infeasible within a reasonable time, a well-recognized challenge in the formal verification of distributed protocols [28, 50, 51, 64, 71]. To address this, we utilize Maude’s bounded search, which explores the state space with depth limits (shown in [square brackets] in Table 4). This enables model checking up to a specified bound. Within the explored bounds, Maude reports no counterexamples to the four properties.